

claude-windows / 50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a\subagents\workflows\wf_ddb0b3fd-b40\agent-a677d83f76698d170.jsonl`
- SHA-256: `3a60b62a79144eabcba2be39e0e430fc37af1ee36dc0d35fd82ba051f70f68b5`
- Source modified: `2026-07-07T04:10:28+00:00`
- Imported at: `2026-07-08T16:00:26+00:00`
- project: `wf_ddb0b3fd-b40`
- session_id: `50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a`
```

Transcript

1. user (2026-07-07T04:01:30.638Z)

You are reviewing an UNCOMMITTED working diff implementing issue #506 (GPU-resident WASB inference) in the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer. The full diff is at C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/50e27c80-8f59-4d8f-8a14-fdbddb9fbb3a/scratchpad/506_diff.patch (read it). The new test file tests/test_wasb_nvdec_resident.py is untracked (not in the diff) ? read it directly from the repo.

Ground truth you must compare against:

- The VALIDATED reference implementation: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/scratch/gpu_resident/exp_t2_full.py (the GpuFrontend class + its grid build ARE what was validated on a real L4: F1 0.574 ? baseline 0.582 on the hard clip). The production port must be semantically faithful to it.
- WASB-SBDT interfaces (not in this repo; trust these facts):
TracknetV2Detector.run_tensor(imgs, affine_mats) moves imgs .to('cuda') itself and calls postprocessor.run(preds, affine_mats) which does affine_mats[scale].cpu().numpy() then indexes [i] per batch item; ImageDataset (eval) returns (imgs_t, hms_t, trans_outputs_inv, xys, visis, img_paths) where trans_outputs_inv is {scale: np.float64 [2,3]}; per-frame preprocessing is PIL RGB -> cv2.warpAffine(trans_input, (512,288), INTER_LINEAR) -> ToTensor -> Normalize(ImageNet); get_transform uses c=(w/2,h/2), s=max(h,w); OnlineTracker uses np.Inf; WASB detector.py calls torch.load(model_path) with no weights_only arg.
- The CPU decode path (run(), run_video_streaming) must be BEHAVIOR-UNCHANGED on the WSL torch-1.11 + numpy<1.24 env (byte-parity guarantee), except that torch>=2.6 envs get the weights_only + np.Inf shims.

Report ONLY defects you can ground in the actual code you read ? cite file:line. No style nits, no speculative maybes without a concrete failure scenario. Return findings as structured output.

DIMENSION: config/runner plumbing + failure modes. Scrutinize backend/config/models.py (WasbIndexConfig decode_backend + validator interplay with device field default; does the validator break any existing valid config?), backend/pipeline/detectors/native_wasb_runner.py (argv construction order/uniqueness, healthcheck gates, the run_predict early-abort ? does returning None there leak anything or skip needed cleanup? env override precedence, does WASB_DECODE_BACKEND env var get validated anywhere?), wasb_infer.py main() (branch ordering, arg interactions e.g. --stream-video AND --decode-backend nvdec_resident together, --limit/--batch-size threading), and the WSL runner (wasb_runner.py ? read it: does it silently ignore decode_backend? is that consistent with the config comment claims?). Also check _torch_load_compat for thread-safety/reentrancy issues in this codebase's actual usage.

2. user (2026-07-07T04:01:37.498Z)

<system-reminder>[Truncated: PARTIAL view ? showing lines 1-875 of 1092 total (26506 tokens, cap 25000). Call Read with offset=876 limit=875 for the next page, or Grep to find a specific section. Do NOT answer from this page alone if the answer may be further in the file.]</system-reminder>

```

1      diff --git a/backend/config/models.py b/backend/config/models.py
2      index cd6f2fc..a7c997c 100644
3      --- a/backend/config/models.py
4      +++ b/backend/config/models.py
5      @@ -119,6 +119,28 @@ class WasbIndexConfig(BaseModel):
6          # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
7          # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
8          prefetch_batches: Optional[int] = None
9      + # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
10     + # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
11     + # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
12     + # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
13     + # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
14     + # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
15     + # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
16     + # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
17     + # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
18     + decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
19     +
20     + @model_validator(mode="after")
21     + def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
22     +     # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
23     +     # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
24     +     # healthcheck hard-fails if auto resolves to cpu with this backend on.
25     +     if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
26     +         raise ValueError(
27     +             f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
28     +             f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
29     +         )
30     +     return self
31
32     @field_validator("batch_size")
33     @classmethod
34     diff --git a/backend/pipeline/detectors/native_wasb_runner.py
b/backend/pipeline/detectors/native_wasb_runner.py
35     index 54cec69..230d889 100644
36     --- a/backend/pipeline/detectors/native_wasb_runner.py
37     +++ b/backend/pipeline/detectors/native_wasb_runner.py
38     @@ -60,6 +60,10 @@ _CUDA_PROBE_CODE = (
39     +     "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"
40     + )
41
42     + # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
43     + # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2
mid-run.
44     + _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
45     +
46
47     @dataclass
48     class NativeWasbConfig:
49     @@ -90,6 +94,10 @@ class NativeWasbConfig:

```

```

50         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
51         batch_size: int = 0
52         prefetch_batches: int = 0
53     +     # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
54     +     # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
55     +     # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
56     +     decode_backend: str = "cpu"
57
58     @classmethod
59     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
60     @@ -115,6 +123,7 @@ class NativeWasbConfig:
61         stream_video=(True if sv is None else bool(sv)),
62         batch_size=int(w.batch_size or 0),
63         prefetch_batches=int(w.prefetch_batches or 0),
64     +         decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
65     )
66
67
68     @@ -203,6 +212,11 @@ class NativeWasbRunner(DetectorRunner):
69         return False, (
70             f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)})."
71         )
72     +     if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
73     +         return False, (
74     +             f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
75     +             f"{'', '.join(_VALID_DECODE_BACKENDS)})."
76     +         )
77     +     try:
78     +         res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
79     +     except FileNotFoundError:
80     @@ -231,6 +245,14 @@ class NativeWasbRunner(DetectorRunner):
81         "inference will be slow. Set indexing.wasb.device=cuda to require a
GPU.",
82         out,
83     )
84     +     # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
the
85     +     # config validator already rejects) cannot run the GPU-resident path ? fail
loud
86     +     # instead of letting wasb_infer.py die mid-run on a paid box.
87     +     if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
88     +         return False, (
89     +             f"decode_backend=nvdec_resident requires CUDA, but the effective device
is "
90     +             f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
91     +         )
92     +     return True, out
93
94     def run_predict(
95     @@ -263,7 +285,12 @@ class NativeWasbRunner(DetectorRunner):
96         os.path.abspath(str(video_win_path)),
97     ]
98     frames_dir: Optional[str] = None
99     -     if self.cfg.stream_video:
100     +     if self.cfg.decode_backend == "nvdec_resident":
101     +         # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
itself
102     +         # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
neither
103     +         # --stream-video nor --frames_out_dir applies and there is no frame cache

```

```

to clean.
104     +         argv += ["--decode-backend", self.cfg.decode_backend]
105     +         elif self.cfg.stream_video:
106     +             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
107     +             argv.append("--stream-video")
108     +         else:
109     +             @@ -273,6 +300,15 @@ class NativeWasbRunner(DetectorRunner):
110     +                 # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
111     +                 # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
112     +                 device = self._effective_device()
113     +                 if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
114     +                     # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu
on a
115     +                     # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
subprocess.
116     +                     logger.error(
117     +                         "decode_backend=nvdec_resident requires CUDA, but the effective device
is "
118     +                         "%r ? aborting before the WASB subprocess.",
119     +                         device,
120     +                     )
121     +                     return None
122     +                 argv += [
123     +                     "--weights",
124     +                     weights,
125     +                     diff --git a/backend/pipeline/detectors/wasb_infer.py
b/backend/pipeline/detectors/wasb_infer.py
126     +                     index cbe1416..f41427b 100644
127     +                     --- a/backend/pipeline/detectors/wasb_infer.py
128     +                     +++ b/backend/pipeline/detectors/wasb_infer.py
129     +                     @@ -30,7 +30,16 @@ resume mid-stream). We exploit that split:
130
131     +                     The cache is keyed by the output path (`<out>_wasbcache/` by default), lives next
132     +                     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
133     +                     -weights, sport, window size, or frame count change.
134     +                     +weights, sport, window size, frame count, device, or decode backend change.
135     +                     +
136     +                     +GPU-resident decode (#506)
137     +                     +-----
138     +                     +`--decode-backend nvdec_resident` (default `cpu`) moves decode + resize + normalize
139     +                     +onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
140     +                     +`grid_sample` resize replicating WASB's `cv2.warpAffine` exactly ? GPU ImageNet
141     +                     +normalize ? detector. Validated on a real L4 (issue #506): ~1.7x the cv2 streaming
142     +                     +path with rally F1 preserved on the hard golden clip. CUDA-only; needs the torch-2.x +
143     +                     +PyNvVideoCodec env (the WSL torch-1.11 env keeps `cpu`, its behaviour untouched).
144
145     +                     Usage (in WSL, from ~/models/WASB-SBDT/src):
146     +                     python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
147     +                     @@ -53,6 +62,16 @@ CACHE_VERSION = 1
148     +                     _DET_FILE = "det_raw.jsonl"
149     +                     _MANIFEST_FILE = "manifest.json"
150
151     +                     + # Pixel-decode backends (#506). "cpu" = the historical cv2 paths (disk frames or
152     +                     + # --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
153     +                     + # resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
154     +                     + DECODE_BACKENDS = ("cpu", "nvdec_resident")
155     +                     +
156     +                     + # ImageNet statistics ? MUST match WASB's build_img_transforms Normalize (the GPU-
resident
157     +                     + # frontend normalizes on-device instead of through torchvision's transform).
158     +                     + IMAGENET_MEAN = (0.485, 0.456, 0.406)

```

```

159 +IMAGENET_STD = (0.229, 0.224, 0.225)
160 +
161
162 def _build_cfg(weights: str, sport: str, device: str = "cuda"):
163     """Compose the WASB eval config via Hydra, overriding model/weights/device.
164 @@ -85,6 +104,48 @@ def _frame_id(path: str) -> int:
165         return int(digits) if digits else -1
166
167
168 +# ----- #
169 +# Runtime-compat shims for the torch-2.x / NumPy-2.x env (#506). Both are exact
170 +# no-ops on the verified WSL torch-1.11 + numpy<1.24 stack.
171 +# ----- #
172 +def _ensure_np_inf() -> None:
173 +    """NumPy 2.0 removed the ``np.Inf`` alias, which WASB's OnlineTracker still reads.
174 +    Restore it before any tracker use; a no-op on NumPy 1.x."""
175 +    import numpy as np
176 +
177 +    if not hasattr(np, "Inf"):
178 +        np.Inf = np.inf # type: ignore[attr-defined]
179 +
180 +
181 +@contextmanager
182 +def _torch_load_compat():
183 +    """Scope ``torch.load`` to ``weights_only=False`` while WASB builds its detector.
184 +
185 +    WASB's ``detector.py`` calls ``torch.load(model_path)`` with no ``weights_only``
186 +    argument; torch 2.6 flipped that default to True, which rejects the (pre-2.6,
187 +    owner-staged, trusted) WASB checkpoint. The kwarg is injected only when the
188 +    installed torch has it AND the caller didn't pass it, so torch 1.11 (no such
189 +    kwarg) and explicit callers are untouched."""
190 +    import inspect
191 +
192 +    import torch
193 +
194 +    real_load = torch.load
195 +    if "weights_only" not in inspect.signature(real_load).parameters:
196 +        yield # torch < 1.13 ? nothing to shim
197 +        return
198 +
199 +    def _load(*args, **kwargs):
200 +        kwargs.setdefault("weights_only", False)
201 +        return real_load(*args, **kwargs)
202 +
203 +    torch.load = _load
204 +    try:
205 +        yield
206 +    finally:
207 +        torch.load = real_load
208 +
209 +
210 +# ----- #
211 +# Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
212 +# ----- #
213 @@ -151,6 +212,7 @@ def manifest_compatible(
214     frames_in: int,
215     frames_total: int,
216     device: str = "cuda",
217 +    decode_backend: str = "cpu",
218 ) -> bool:
219     """A cache is reusable only if the run that produced it matches this run's inputs.
220
221 @@ -158,6 +220,11 @@ def manifest_compatible(
222     field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
cache

```

```

223         (detections can differ numerically across devices), and vice-versa.
224
225     +     ``decode_backend`` follows the same pattern (#506): pre-existing caches (no field)
226     +     are ``cpu``-decoded, so they stay compatible with cpu runs ? but an
227     +     ``nvdec_resident``
228     +     run never reuses a cpu cache (NVDEC decode + the GPU affine warp are trajectory-
229     +     level,
230     +     not byte-level, equivalents), and vice-versa.
231     +
232     +     ``frames_dir`` is the **already-canonical cache key** the caller stored in the
233     +     manifest
234     +     (``run()`` passes its ``cache_key`` here, and writes the same value into the
235     +     manifest):
236     +     ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
237     +     ``video:<abspath>``
238
239     @@ -174,6 +241,7 @@ def manifest_compatible(
240         and manifest.get("frames_in") == frames_in
241         and manifest.get("frames_total") == frames_total
242         and manifest.get("device", "cuda") == device
243         +         and manifest.get("decode_backend", "cpu") == decode_backend
244     )
245
246     @@ -487,17 +555,11 @@ def run(
247         from dataloaders import build_img_transforms, build_seq_transforms
248         from dataloaders.dataset_loader import ImageDataset
249         from torch.utils.data import DataLoader
250
251         -
252         -     # NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
253         -     the
254         -     # `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
255         -     suppresses
256         -     # the mypy false positive where the main env resolves `trackers` to the Roboflow
257         -     package
258         -     # (which has no `build_tracker`). There is NO runtime collision ? this runs in a
259         -     separate
260         -     # subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
261         -     dep.
262         -     from trackers import build_tracker # type: ignore[attr-defined]
263         -     from utils import Center
264
265         from detectors import build_detector
266
267         +     _ensure_np_inf() # NumPy-2.x env: the tracker pass reads np.Inf
268         +     streaming = frame_loader is not None
269
270         cfg = _build_cfg(weights, sport, device)
271
272     @@ -550,6 +612,7 @@ def run(
273         frames_in=frames_in,
274         frames_total=len(frames),
275         device=device,
276         +         decode_backend="cpu", # run() IS the cpu decode path (disk frames / cv2
277         +         stream)
278     )
279     if resume:
280         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
281
282     @@ -570,6 +633,7 @@ def run(
283         "frames_in": frames_in,
284         "frames_total": len(frames),
285         "device": device,
286         +         "decode_backend": "cpu",
287         +         "windows_total": windows_total,
288         +         "windows_done": windows_done,
289     }
290
291     @@ -598,7 +662,8 @@ def run(

```

```

277         loader = DataLoader(
278             ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
279         )
280         detector = build_detector(cfg)
281         with _torch_load_compat(): # torch-2.6 env: WASB's torch.load needs
weights_only=False
282             detector = build_detector(cfg)
283
284         from contextlib import ExitStack
285
286     @@ -682,6 +747,25 @@ def run(
287         logger.info("detector cache already complete -> tracker-only re-run")
288
289         # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
290         + return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
291         +
292         +
293     +def _tracker_pass_and_write(cfg, det_by_fid, out_csv: str) -> str:
294         + """Tracker re-run over the full ordered detection cache ? trajectory CSV.
295         +
296         + Shared verbatim by every decode backend: the tracker consumes the same per-frame
297         + candidate lists regardless of how the pixels were decoded, so keeping this single
298         + keeps the output contract single.
299         +
300         + NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside
the
301         + `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
302         + the mypy false positive where the main env resolves `trackers` to the Roboflow
package
303         + (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
304         + subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
305         + """
306         + from trackers import build_tracker # type: ignore[attr-defined]
307         +
308         + _ensure_np_inf() # OnlineTracker reads np.Inf (removed in NumPy 2.0)
309         + tracker = build_tracker(cfg)
310         + tracker.refresh()
311         + rows = []
312     @@ -832,6 +916,320 @@ def run_video_streaming(
313         release()
314
315
316     +## ----- #
317     +## GPU-resident decode backend (#506): NVDEC decode ? affine grid_sample resize ?
318     +## normalize ? detector, all on the GPU (no host round-trip). Validated on a real
319     +## L4: ~1.7x the cv2 streaming path with rally F1 preserved (0.574 ? baseline 0.582
320     +## on GX010128 at the SERVED preset).
321     +## ----- #
322     +def build_affine_grid(trans_inv, inp_w: int, inp_h: int, orig_w: int, orig_h: int):
323         + """`F.grid_sample` sampling grid replicating `cv2.warpAffine(img, trans,
324         + (inp_w, inp_h), INTER_LINEAR)` ? THE #506 lesson lives here.
325         +
326         + The GPU resize MUST be WASB's affine warp (center/scale geometry + cv2's
327         + integer-coordinates-are-pixel-centers convention), NOT a plain resize:
328         + ``F.interpolate``'s half-pixel mapping shifts every sample by a constant
329         + sub-pixel offset, which tips the sigmoid past ``score_threshold`` on marginal
330         + frames and silently over-detects ? F1 0.58 ? 0.07 on the hard clip GX010128,
331         + while the easy clip masked it at 98% agreement. Locked by a cv2-parity unit test.
332         +
333         + ``trans_inv`` is WASB's INVERSE affine (`get_affine_transform(..., inv=1)`),
334         + mapping output px ? source px; each source position is normalized to [-1, 1]
335         + via ``gx = (2*x + 1)/W - 1`` (cv2's half-pixel convention). Pure torch+numpy ?

```

```

336 + no WASB imports ? so the parity test runs on any CPU box.
337 +
338 + Returns a ``[1, inp_h, inp_w, 2]`` float32 grid for ``F.grid_sample(...,
339 + mode='bilinear', align_corners=False, padding_mode='border')``.
340 + """
341 + import numpy as np
342 + import torch
343 +
344 + ys, xs = torch.meshgrid(
345 +     torch.arange(inp_h, dtype=torch.float32),
346 +     torch.arange(inp_w, dtype=torch.float32),
347 +     indexing="ij",
348 + )
349 + out_coords = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3] output
px
350 + mt = torch.from_numpy(np.ascontiguousarray(trans_inv, dtype=np.float32)) # [2,3]
351 + src = out_coords @ mt.transpose(0, 1) # [H,W,2] source px
352 + gx = (2.0 * src[..., 0] + 1.0) / orig_w - 1.0
353 + gy = (2.0 * src[..., 1] + 1.0) / orig_h - 1.0
354 + return torch.stack([gx, gy], dim=-1).unsqueeze(0) # [1,H,W,2]
355 +
356 +
357 +def _wasb_inverse_affines(orig_h: int, orig_w: int, out_wh, out_scales):
358 +     """Per-scale inverse affines exactly as WASB's ``ImageDataset`` builds
359 +     ``trans_outputs_inv`` (``get_transform(img, wh, inv=1)``: center = frame middle,
360 +     scale = max(h, w), output size halved per extra scale). Returns
361 +     ``{scale: np.ndarray[2, 3] float64}`` ? float64 like ``cv2.getAffineTransform``,
362 +     matching what the DataLoader path collates for the postprocessor."""
363 +     import numpy as np
364 +     from utils.image import get_affine_transform
365 +
366 +     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
367 +     s = max(orig_h, orig_w) * 1.0
368 +     out = {}
369 +     out_w, out_h = int(out_wh[0]), int(out_wh[1])
370 +     for scale in out_scales:
371 +         out[scale] = get_affine_transform(c, s, 0, [out_w, out_h], inv=1)
372 +         out_w //= 2
373 +         out_h //= 2
374 +     return out
375 +
376 +
377 +class NvdecResidentFrontend:
378 +     """On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
379 +
380 +     PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
381 +     NVDEC engine; frames arrive as CUDA tensors via DLPack and never touch the host.
382 +     ``read_chw`` then applies the cv2-faithful affine warp (``build_affine_grid``)
383 +     and ImageNet-normalizes on-device, yielding the same ``[3, inp_h, inp_w]``
384 +     float32 tensor per frame that the ImageDataset path assembles on the CPU.
385 +     """
386 +
387 +     def __init__(self, video_path: str, inp_wh, gpu_id: int = 0):
388 +         import PyNvVideoCodec as nvc
389 +         import torch
390 +
391 +         self._torch = torch
392 +         self._device = torch.device("cuda", gpu_id)
393 +         inp_w, inp_h = int(inp_wh[0]), int(inp_wh[1])
394 +         try:
395 +             self._dec = nvc.SimpleDecoder(
396 +                 video_path, gpu_id=gpu_id, output_color_type=nvc.OutputColorType.RGB
397 +             )
398 +         except Exception as e:
399 +             raise RuntimeError(

```



```

400 +         f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
401 +         f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
402 +         f"and libnvcuvid/libnvidia-encode on the box
(deploy/cloudrun/Dockerfile "
403 +         f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
404 +     ) from e
405 +     self.count = len(self._dec)
406 +     probe = self._decode_rgb_hwc(0)
407 +     self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
408 +     trans_inp_inv = _wasb_inverse_affines(
409 +         self.orig_h, self.orig_w, (inp_w, inp_h), [0]
410 +     )[0]
411 +     self._grid = build_affine_grid(
412 +         trans_inp_inv, inp_w, inp_h, self.orig_w, self.orig_h
413 +     ).to(self._device)
414 +     self._mean = torch.tensor(IMAGENET_MEAN, device=self._device).view(3, 1, 1)
415 +     self._std = torch.tensor(IMAGENET_STD, device=self._device).view(3, 1, 1)
416 +
417 +     def _decode_rgb_hwc(self, index: int):
418 +         """Frame ``index`` as an on-GPU HWC RGB uint8 tensor (zero-copy via DLPack)."""
419 +         torch = self._torch
420 +         t = torch.from_dlpack(self._dec[index])
421 +         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
422 +             t = t.permute(1, 2, 0) # CHW layout variant ? HWC
423 +         if t.dim() != 3 or t.shape[-1] != 3:
424 +             raise RuntimeError(
425 +                 f"unexpected NVDEC frame shape {tuple(t.shape)} for frame {index} ? "
426 +                 f"expected 3-channel RGB (is output_color_type=RGB supported here?)"
427 +             )
428 +         return t
429 +
430 +     def read_chw(self, index: int):
431 +         """Decoded frame ``index`` ? the normalized ``[3, inp_h, inp_w]`` float32 CUDA
432 +         tensor the detector expects (warpAffine-faithful resize + ImageNet
433 +         normalize)."""
434 +         import torch.nn.functional as F
435 +         hwc = self._decode_rgb_hwc(index)
436 +         chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
437 +         warped = F.grid_sample(
438 +             chw, self._grid, mode="bilinear", align_corners=False,
439 +             padding_mode="border"
440 +         ).squeeze(0)
441 +         return (warped - self._mean) / self._std
442 +
443 +     def release(self):
444 +         self._dec = None
445 +
446 +     +def run_video_nvdec_resident(
447 +         video_path: str,
448 +         weights: str,
449 +         out_csv: str,
450 +         sport: str = "badminton",
451 +         limit: int = 0,
452 +         batch_size: int = 8,
453 +         log_every_batches: int = 50,
454 +         cache_dir: "str | None" = None,
455 +         flush_every: int = 1000,
456 +         fresh: bool = False,
457 +     ) -> str:
458 +         """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
459 +         warped+normalized frames straight to the detector ? decode, resize, and
460 +         normalize never touch the host. CUDA-only by construction.
461 +

```

```

462 + NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
463 + warp differ from cv2 at float precision) ? equivalence is trajectory-level
464 + (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
465 + rally-F1-level (the golden gate), so the detection cache is keyed by
466 + ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
467 + cache + tracker tail as ``run()`` otherwise.
468 + """
469 + import time
470 +
471 + import numpy as np
472 + import torch
473 +
474 + from detectors import build_detector
475 +
476 + _ensure_np_inf()
477 + if not torch.cuda.is_available():
478 +     raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
479 +
480 + cfg = _build_cfg(weights, sport, "cuda")
481 + frames_in = int(cfg["model"]["frames_in"])
482 + input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
483 + output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
484 + out_scales = list(cfg["model"]["out_scales"])
485 +
486 + fe = NvdecResidentFrontend(video_path, input_wh)
487 + try:
488 +     total = min(fe.count, limit) if limit else fe.count
489 +     if total < frames_in:
490 +         raise SystemExit(f"need >= {frames_in} frames, video has {total}")
491 +     windows_total = total - frames_in + 1
492 +     logger.info(
493 +         f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
494 +         f"{windows_total} windows"
495 +     )
496 +
497 +     # ---- cache setup / resume decision (mirrors run(); keyed nvdec_resident) ---
498 +     #
499 +     if cache_dir is None:
500 +         cache_dir = default_cache_dir(out_csv)
501 +         os.makedirs(cache_dir, exist_ok=True)
502 +         det_jsonl, _ = _cache_paths(cache_dir)
503 +         cache_key = "video:" + osp.abspath(video_path)
504 +         manifest = None if fresh else read_manifest(cache_dir)
505 +         resume = manifest_compatible(
506 +             manifest or {},
507 +             frames_dir=cache_key,
508 +             weights=weights,
509 +             sport=sport,
510 +             frames_in=frames_in,
511 +             frames_total=total,
512 +             device="cuda",
513 +             decode_backend="nvdec_resident",
514 +         )
515 +         if resume:
516 +             windows_done, det_by_fid = load_and_clean_cache(cache_dir)
517 +             logger.info(
518 +                 f"resuming from cache: {windows_done}/{windows_total} windows already
519 +                 detected"
520 +             )
521 +         else:
522 +             if manifest is not None:
523 +                 logger.info("cache incompatible with this run -> starting fresh")
524 +                 reset_cache(cache_dir)
525 +                 windows_done, det_by_fid = 0, defaultdict(list)

```

```

525 +     base_manifest = {
526 +         "version": CACHE_VERSION,
527 +         "frames_dir": cache_key,
528 +         "weights": weights,
529 +         "sport": sport,
530 +         "frames_in": frames_in,
531 +         "frames_total": total,
532 +         "device": "cuda",
533 +         "decode_backend": "nvdec_resident",
534 +         "windows_total": windows_total,
535 +         "windows_done": windows_done,
536 +     }
537 +     write_manifest(cache_dir, base_manifest)
538 +
539 +     # ---- GPU-resident detector pass over the un-cached windows -----
540 +     #
541 +     if windows_done < windows_total:
542 +         with _torch_load_compat():
543 +             detector = build_detector(cfg)
544 +             # {scale: [1,2,3] float64 CPU tensor} ? the postprocessor .cpu().numpy()'s
545 +             # these, so CPU float64 matches the DataLoader-collated path exactly.
546 +             trans_t = {
547 +                 s: torch.from_numpy(np.ascontiguousarray(m)).unsqueeze(0)
548 +                 for s, m in _wasb_inverse_affines(
549 +                     fe.orig_h, fe.orig_w, output_wh, out_scales
550 +                 ).items()
551 +             }
552 +             t0 = time.time()
553 +             done = windows_done
554 +             win_idx = windows_done
555 +             n_batches = 0
556 +             log_every = max(1, log_every_batches)
557 +             flush_n = max(1, flush_every)
558 +             pending = []
559 +             window = [] # sliding [(fid, chw)] of the last frames_in decoded frames
560 +             batch_fids, batch_imgs = [], []
561 +             logger.info(
562 +                 f"detecting on {windows_total - windows_done} remaining windows "
563 +                 f"(total {windows_total}, batch={batch_size}, source=nvdec-
564 +                 resident)..."
565 +             )
566 +             det_f = open(det_jsonl, "a")
567 +             try:
568 +                 with torch.no_grad():
569 +                     # Window i needs frames [i, i+frames_in] ? start decoding at the
570 +                     # first un-cached window's first frame (NVDEC seeks past the
571 +                     # prefix).
572 +                     for idx in range(windows_done, total):
573 +                         window.append((idx, fe.read_chw(idx)))
574 +                         if len(window) > frames_in:
575 +                             window.pop(0)
576 +                         if len(window) < frames_in:
577 +                             continue
578 +                         batch_fids.append([fid for fid, _ in window])
579 +                         batch_imgs.append(
580 +                             torch.cat([chw for _, chw in window], dim=0)
581 +                         )
582 +                         if len(batch_imgs) < batch_size and idx < total - 1:
583 +                             continue
584 +                         nb = len(batch_imgs)
585 +                         imgs = torch.stack(batch_imgs, dim=0)
586 +                         # .repeat (not .expand): the postprocessor calls .numpy() on
587 +                         # these.
588 +                         affine_mats = {
589 +                             s: m.repeat(nb, 1, 1) for s, m in trans_t.items()

```

```

586     +         }
587     +         batch_results, _ = detector.run_tensor(imgs, affine_mats)
588     +         for ib in range(nb):
589     +             frames_payload = []
590     +             for ie in sorted(batch_results[ib].keys()):
591     +                 fid = batch_fids[ib][ie]
592     +                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
593     +                 frames_payload.append([fid, plain])
594     +                 det_by_fid[fid].extend(plain)
595     +                 pending.append({"w": win_idx, "f": frames_payload})
596     +                 win_idx += 1
597     +         done += nb
598     +         n_batches += 1
599     +         batch_fids, batch_imgs = [], []
600     +         if len(pending) >= flush_n or done >= windows_total:
601     +             _append_windows(det_f, pending)
602     +             pending = []
603     +             base_manifest["windows_done"] = done
604     +             write_manifest(cache_dir, base_manifest)
605     +         if n_batches % log_every == 0 or done >= windows_total:
606     +             el = time.time() - t0
607     +             new_done = done - windows_done
608     +             rate = new_done / el if el > 0 else 0.0
609     +             eta = (windows_total - done) / rate if rate > 0 else 0.0
610     +             logger.info(
611     +                 f"{done}/{windows_total} "
612     +                 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
613     +                 f"elapsed {el:.0f}s | ETA {eta:.0f}s"
614     +             )
615     +         if pending: # safety net; the done>=windows_total branch already
flushed
616     +             _append_windows(det_f, pending)
617     +             base_manifest["windows_done"] = done
618     +             write_manifest(cache_dir, base_manifest)
619     +         finally:
620     +             det_f.close()
621     +             logger.info(f"detection done in {time.time() - t0:.0f}s; running
tracker...")
622     +         else:
623     +             logger.info("detector cache already complete -> tracker-only re-run")
624     +         finally:
625     +             fe.release()
626     +
627     +         return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
628     +
629     +
630     def main():
631         ap = argparse.ArgumentParser()
632         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
633     @@ -847,6 +1245,16 @@ def main():
634         help="FAST PATH: decode --video on the fly and feed frames straight to "
635         "the detector (no PNG extraction). Output-equivalent to the disk path.",
636     )
637     + ap.add_argument(
638     +     "--decode-backend",
639     +     default="cpu",
640     +     choices=list(DECODE_BACKENDS),
641     +     help="cpu (default) = the historical cv2 decode paths (disk frames or "
642     +     "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
643     +     "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
644     +     "~1.7x on an L4. Requires --video; detections cache separately from cpu "
645     +     "(trajectory-level, not byte-level, equivalence).",
646     + )
647     + ap.add_argument("--weights", required=True)

```

```

648         ap.add_argument("--out", required=True)
649         ap.add_argument("--sport", default="badminton")
650     @@ -887,6 +1295,30 @@ def main():
651         )
652         args = ap.parse_args()
653
654     +     if args.decode_backend == "nvdec_resident":
655     +         # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither
the
656     +         # --stream-video cv2 path nor the extract-frames fallback below may run.
657     +         if not args.video:
658     +             raise SystemExit("--decode-backend nvdec_resident requires --video")
659     +         if args.device != "cuda":
660     +             raise SystemExit(
661     +                 "--decode-backend nvdec_resident requires --device cuda "
662     +                 "(NVDEC decode runs on the GPU)"
663     +             )
664     +         run_video_nvdec_resident(
665     +             args.video,
666     +             args.weights,
667     +             args.out,
668     +             args.sport,
669     +             args.limit,
670     +             batch_size=args.batch_size,
671     +             log_every_batches=args.log_every_batches,
672     +             cache_dir=args.cache_dir,
673     +             flush_every=args.flush_every,
674     +             fresh=args.fresh,
675     +         )
676     +         return
677     +
678     if args.stream_video:
679         if not args.video:
680             raise SystemExit("--stream-video requires --video")
681 diff --git a/deploy/cloudrun/Dockerfile b/deploy/cloudrun/Dockerfile
682 index 836ffb7..aee76a0 100644
683 --- a/deploy/cloudrun/Dockerfile
684 +++ b/deploy/cloudrun/Dockerfile
685 @@ -8,13 +8,33 @@
686     # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
687     # native GPU box, deploy/digitalocean/gpu_setup.sh):
688     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
689     -# * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
690     -# (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
691     +# * a conda `wasb` env (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? the #506
modernized stack,
692     +# validated on a real L4: cpu-decode F1 0.576 ? the torch-1.11 baseline 0.582) runs
the WASB
693     +# shuttle detector as a subprocess (the native runner shells out to $WASB_PYTHON),
so the app's
694     +# torch never co-installs with it.
695     #
696     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance
NOT cleared for
697     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
698     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-
owner serving.
699
700     +# Driver userspace libs for NVDEC/NVENC (#506): GPU hosts inject the COMPUTE driver
stack but not
701     +# the video-codec libs ? no libnvcuvid.so.1, no libnvidia-encode.so.1 ? and

```

PyNvVideoCodec needs

```
702     ## BOTH (it initializes NVENC even for decode-only). Extract the pair from the matching
datacenter
703     ## driver .run in a throwaway stage. Keep NVIDIA_DRIVER_VERSION aligned with the GPU
fleet's driver
704     ## (580.159.03 = the validated L4 combo); these userspace libs talk to the host-injected
libcuda,
705     ## so a large version skew can break decode ? the golden gate on the real image is the
check.
706     +ARG NVIDIA_DRIVER_VERSION=580.159.03
707     +FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04 AS nvlibs
708     +ARG NVIDIA_DRIVER_VERSION
709     +RUN apt-get update && apt-get install -y --no-install-recommends curl ca-certificates \
710     +     && rm -rf /var/lib/apt/lists/* \
711     +     && curl -fsSL -o /tmp/drv.run \
712     +     "https://us.download.nvidia.com/tesla/${NVIDIA_DRIVER_VERSION}/NVIDIA-
Linux-x86_64-${NVIDIA_DRIVER_VERSION}.run" \
713     +     && sh /tmp/drv.run --extract-only --target /tmp/drv \
714     +     && mkdir -p /nvlibs \
715     +     && cp -a /tmp/drv/libnvcuvid.so.* /tmp/drv/libnvidia-encode.so.* /nvlibs/ \
716     +     && rm -rf /tmp/drv /tmp/drv.run
717     +
718     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
719
720     ENV DEBIAN_FRONTEND=noninteractive \
721     @@ -29,22 +49,38 @@ RUN apt-get update && apt-get install -y --no-install-recommends \
722         python3 python3-pip python3-venv \
723         && rm -rf /var/lib/apt/lists/*
724
725     ## NVDEC/NVENC userspace libs (#506, from the nvlibs stage above) + the .so.1 sonames
726     ## PyNvVideoCodec dlopens. Harmless when decode_backend=cpu; required for
nvdec_resident.
727     +ARG NVIDIA_DRIVER_VERSION
728     +COPY --from=nvlibs /nvlibs/ /usr/lib/x86_64-linux-gnu/
729     +RUN ln -sf "libnvcuvid.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvcuvid.so.1 \
730     +     && ln -sf "libnvidia-encode.so.${NVIDIA_DRIVER_VERSION}" /usr/lib/x86_64-linux-
gnu/libnvidia-encode.so.1 \
731     +     && ldconfig
732     +
733     # --- WASB detector env (micromamba + conda-forge ? BSD-licensed, NO Anaconda ToS; P3a,
ADR-005) ---
734     # Enforces ADR-005 (every dependency ? code, data, weights, AND the build toolchain ?
stays
735     # MIT/Apache/BSD): micromamba (BSD-3) + conda-forge (community) replaces Miniconda's
Anaconda-ToS-gated
736     # `defaults` channels (the ?200-employee Terms-of-Service trap flagged in
PACKAGING_PLAN.md §3).
737     # MAMBA_ROOT_PREFIX=$CONDA_DIR (=/opt/conda) keeps the env at $CONDA_DIR/envs/wasb so
$WASB_PYTHON below
738     -# is UNCHANGED. Same verified torch-1.11.0+cu113 WASB stack; the pip wheels are
unaffected by the switch.
739     ## is UNCHANGED.
740     ##
741     ## #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 + torch
2.6.0+cu124
742     ## (torch ?2 needs py?3.9; WASB HRNet ports clean ? cpu F1 0.576 ? the torch-1.11
baseline 0.582) +
743     ## PyNvVideoCodec (MIT ? the practical NVDEC path; TorchCodec needs a matching ffmpeg
build and could
744     ## not load) for decode_backend=nvdec_resident. wasb_infer.py carries the two version
shims this bump
745     ## needs (torch.load weights_only=False; the NumPy-2.0 np.Inf alias for WASB's tracker).
numpy is
746     ## UNPINNED (the old "numpy<1.24" was the torch-1.11 ABI pin). WASB-SBDT ships no
```

```

requirements.txt ?
747     ## the explicit list below is the (L4-validated) dependency closure its src/ imports
need.
748     ENV MAMBA_ROOT_PREFIX=/opt/conda
749     RUN curl -sSL https://micro.mamba.pm/api/micromamba/linux-64/latest \
750         | tar -xj -C /usr/local bin/micromamba \
751     -    && micromamba create -y -q -n wasb -c conda-forge python=3.8 \
752     +    && micromamba create -y -q -n wasb -c conda-forge python=3.10 \
753         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
754         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
755     -    torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
756     -    && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
757     +    torch==2.6.0 torchvision==0.21.0 --index-url
https://download.pytorch.org/whl/cu124 \
758     +    && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q PyNvVideoCodec \
759     && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
760     -    hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
761     +    hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib numpy
762
763     # --- Main application env ---
764     WORKDIR /app
765     diff --git a/deploy/digitalocean/gpu_setup.sh b/deploy/digitalocean/gpu_setup.sh
766     index 8d47e49..a89cc1b 100755
767     --- a/deploy/digitalocean/gpu_setup.sh
768     +++ b/deploy/digitalocean/gpu_setup.sh
769     @@ -3,8 +3,14 @@
770     # Native (no-WSL) WASB shuttle-detector setup for a Linux GPU box ? the M3.5 enabler
for
771     # real trajectory extraction. Replicates docs/TRACKNET_WSL_SETUP.md's WSL conda env
772     # *natively* on Linux. Run AFTER ./bootstrap.sh --gpu (which provides CUDA torch for
the
773     -# main venv); this builds a SEPARATE conda env for WASB (torch 1.11) ? never co-
installed
774     -# with the indexer's torch (framework conflict).
775     +# main venv); this builds a SEPARATE conda env for WASB ? never co-installed with the
776     +# indexer's torch (framework conflict).
777     +#
778     +# #506 modernized stack (validated end-to-end on a real L4, 2026-07-06): py3.10 +
779     +# torch 2.6.0+cu124 (WASB HRNet ports clean; cpu F1 ? the torch-1.11 baseline) +
780     +# PyNvVideoCodec for the GPU-resident decode backend (indexing.wasb.decode_backend=
781     +# nvdec_resident, ~1.7x). Step [2/6] stages the NVDEC/NVENC driver userspace libs
782     +# PyNvVideoCodec needs (compute-only driver images ship neither).
783     #
784     # Requires an NVIDIA GPU + driver (the DO `gpu-*-base` image ships it).
785     #
786     @@ -18,7 +24,7 @@ CONDA_DIR="${CONDA_DIR:-$HOME/miniconda3}"
787     WASB_REPO="$MODELS_DIR/WASB-SBDT"
788     mkdir -p "$MODELS_DIR"
789
790     -echo "[gpu] [1/5] miniconda (native Linux)"
791     +echo "[gpu] [1/6] miniconda (native Linux)"
792     if [ ! -x "$CONDA_DIR/bin/conda" ]; then
793         curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh
794         bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh
795     @@ -26,29 +32,49 @@ fi
796     # shellcheck disable=SC1091
797     source "$CONDA_DIR/etc/profile.d/conda.sh"
798
799     +echo "[gpu] [2/6] NVDEC/NVENC driver userspace libs (#506: PyNvVideoCodec needs BOTH)"
800     +# Compute-only driver images (GCE DLVM, DO gpu-*-base) ship neither libnvcuvid.so.1 nor

```

```

801     ## libnvidia-encode.so.1. Extract them from the .run matching the RUNNING driver so the
802     ## userspace/kernel versions can never skew (see the #506 / gpu-vm-setup-gotchas notes).
803     if [ ! -f /usr/lib/x86_64-linux-gnu/libnvcuvid.so.1 ] || [ ! -f /usr/lib/x86_64-linux-
804     gnu/libnvidia-encode.so.1 ]; then
805         + DRV="$(nvidia-smi --query-gpu=driver_version --format=csv,noheader | head -1)"
806         + echo "[gpu]          extracting libnvcuvid + libnvidia-encode from driver $DRV ..."
807         + curl -fsSL -o /tmp/drv.run "https://us.download.nvidia.com/tesla/$DRV/NVIDIA-
808         Linux-x86_64-$DRV.run" \
809         || curl -fsSL -o /tmp/drv.run
810         "https://us.download.nvidia.com/XFree86/Linux-x86_64/$DRV/NVIDIA-Linux-x86_64-$DRV.run"
811         + sh /tmp/drv.run --extract-only --target /tmp/drv-extract >/dev/null
812         + sudo cp -a /tmp/drv-extract/libnvcuvid.so.* /tmp/drv-extract/libnvidia-encode.so.*
813         /usr/lib/x86_64-linux-gnu/
814         + ( cd /usr/lib/x86_64-linux-gnu \
815         && sudo ln -sf "libnvcuvid.so.$DRV" libnvcuvid.so.1 \
816         && sudo ln -sf "libnvidia-encode.so.$DRV" libnvidia-encode.so.1 \
817         && sudo ldconfig )
818         + rm -rf /tmp/drv-extract /tmp/drv.run
819     else
820         + echo "[gpu]          already present"
821     +fi
822     +
823     # Modern conda BLOCKS non-interactive `conda create` until the defaults-channel Terms
824     of Service
825     # are accepted (CondaToSNonInteractiveError). Accept them so the env build doesn't fail
826     on a fresh
827     # box. (Observed on a GCP Deep-Learning-VM image, 2026-06-20.)
828     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/main
829     >/dev/null 2>&1 || true
830     conda tos accept --override-channels --channel https://repo.anaconda.com/pkgs/r
831     >/dev/null 2>&1 || true
832
833     -echo "[gpu] [2/5] WASB-SBDT repo"
834     +echo "[gpu] [3/6] WASB-SBDT repo"
835     if [ ! -d "$WASB_REPO/.git" ]; then
836         git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO"
837     fi
838
839     -echo "[gpu] [3/5] conda env 'wasb' (py3.8 + torch 1.11.0+cu113 ? the verified WASB
840     stack)"
841     -conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.8
842     +echo "[gpu] [4/6] conda env 'wasb' (py3.10 + torch 2.6.0+cu124 + PyNvVideoCodec ? #506
843     stack)"
844     +conda env list | grep -q '/envs/wasb$' || conda create -y -q -n wasb python=3.10
845     conda activate wasb
846     -python -m pip install -q torch==1.11.0+cu113 torchvision==0.12.0+cu113 \
847     - --extra-index-url https://download.pytorch.org/whl/cu113
848     -[ -f "$WASB_REPO/requirements.txt" ] && python -m pip install -q -r
849     "$WASB_REPO/requirements.txt" || true
850     -# WASB-SBDT's requirements.txt is INCOMPLETE: src/ imports hydra, pandas, etc. that it
851     omits, so
852     -# `import wasb_infer` deps fail without these. (Found on the GCP L4 port, 2026-06-20.)
853     numpy is
854     -# pinned <1.24 for torch-1.11 ABI compatibility.
855     -python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow
856     scipy tqdm pyyaml matplotlib "numpy<1.24" || true
857     +python -m pip install -q torch==2.6.0 torchvision==0.21.0 \
858     + --index-url https://download.pytorch.org/whl/cu124
859     +python -m pip install -q PyNvVideoCodec
860     ## WASB-SBDT ships NO requirements.txt: src/ imports hydra, pandas, etc. ? this explicit
861     list is
862     ## the L4-validated dependency closure (#506). numpy is UNPINNED (the old "numpy<1.24"
863     was the
864     ## torch-1.11 ABI pin); wasb_infer.py shims the NumPy-2.0 np.Inf removal for WASB's
865     tracker.

```



```

849 +python -m pip install -q hydra-core omegaconf pandas opencv-python-headless pillow
scipy tqdm pyyaml matplotlib numpy || true
850
851 -echo "[gpu] [4/5] pretrained badminton weights"
852 +echo "[gpu] [5/6] pretrained badminton weights"
853 WEIGHTS="$WASB_REPO/pretrained_weights/wasb_badminton_best.pth.tar"
854 if [ -f "$WEIGHTS" ]; then
855     echo "[gpu]         found $WEIGHTS"
856 @@ -56,13 +82,18 @@ else
857     echo "[gpu]         !! MISSING $WEIGHTS ? fetch wasb_badminton_best.pth.tar
(Drive/bucket) before extraction."
858 fi
859
860 -echo "[gpu] [5/5] GPU sanity check (in the wasb env)"
861 +echo "[gpu] [6/6] GPU sanity check (in the wasb env)"
862 python - <<'PY'
863 import torch
864 ok = torch.cuda.is_available()
865 print("[gpu]         torch", torch.__version__, "| cuda_available", ok,
866       "| device", (torch.cuda.get_device_name(0) if ok else "CPU"))
867 assert ok, "CUDA not available in the wasb env ? check the GPU driver / base image"
868 +try:
869 +     import PyNvVideoCodec as pncv
870 +     print("[gpu]         PyNvVideoCodec", getattr(pncv, "__version__", "?"), "OK")
871 +except Exception as e: # nvdec_resident unavailable; the cpu decode path still works
872 +     print("[gpu]         !! PyNvVideoCodec import failed:", repr(e)[:150])
873 PY
874
875     echo "[gpu] native WASB env ready.  Activate with:  source
$CONDA_DIR/etc/profile.d/conda.sh && conda activate wasb"

```

3. user (2026-07-07T04:01:45.161Z)

```

876 diff --git a/docs/CLOUD_GPU_DRYRUN_GUIDE.md b/docs/CLOUD_GPU_DRYRUN_GUIDE.md
877 index 8e263b4..6021064 100644
878 --- a/docs/CLOUD_GPU_DRYRUN_GUIDE.md
879 +++ b/docs/CLOUD_GPU_DRYRUN_GUIDE.md
880 @@ -93,8 +93,9 @@ Watch it finish (look for `SETUP_DONE` and `torch ... cuda_available
True`):
881     gcloud compute ssh rally-gpu --zone="$ZONE" --command='tail -n 20 ~/setup.log'
882     ``
883
884 -> ? You're ready when the log shows the WASB env built with `torch 1.11.0+cu113 |
cuda_available True
885 -> | device NVIDIA L4` and `SETUP_DONE`. (The 5.8 MB detector weights are now staged.)
886 +> ? You're ready when the log shows the WASB env built with `torch 2.6.0+cu124 |
cuda_available True
887 +> | device NVIDIA L4` (plus a `PyNvVideoCodec ... OK` line ? the #506 GPU-resident
decode dep) and
888 +> `SETUP_DONE`. (The 5.8 MB detector weights are now staged.)
889
890 ---
891
892 diff --git a/docs/PACKAGING_PLAN.md b/docs/PACKAGING_PLAN.md
893 index 9822606..5a2873a 100644
894 --- a/docs/PACKAGING_PLAN.md
895 +++ b/docs/PACKAGING_PLAN.md
896 @@ -20,7 +20,7 @@
897 ---
898
899 ## 0. TL;DR
900 -Ship the engine as ONE pip-installable artifact whose behaviour is selected entirely
by the already-merged decoupled-compute seam** (`deployment.profile` / `compute_target` /
Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? not a
frozen monolith, not an Electron/Tauri shell. The hard constraint that forces scope: torch

```

is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a **second**, incompatible* torch env (conda py3.8 + torch 1.11+cu113) shelled out via `\$WASB\_PYTHON`
`[verified native\_wasb\_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, **zero*** torch/GPU) that preserves CUJ-1.5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only** until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config). **LIGHT** is the **FIRST** shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the **FULL** quality path: native WASB + BYO training + in-app annotation. **Delivery is cross-platform** (Windows/macOS/Linux): the same local webserver powers the UI on all three, with **NO** native component** (L9). **Five small engine prerequisites must land before any packaging push** (\$7 P0). The single most actionable finding: **all mutable state is CWD-relative today and** **--setup/server disagree on config.json** ? a silent data-loss footgun that P0 must fix first.

901 +Ship the engine as **ONE** pip-installable artifact whose behaviour is selected entirely by the already-merged decoupled-compute seam** (`deployment.profile` / `compute\_target` / Storage+Compute backends), fronted by a friendly launcher + a Windows installer ? **not** a frozen monolith, **not** an Electron/Tauri shell. The hard constraint that forces scope: torch is deliberately out of `pyproject` (needs `--index-url`) **and** native WASB needs a **second**, isolated* torch env (conda py3.10 + torch 2.6+cu124 + PyNvVideoCodec since #506) shelled out via `\$WASB\_PYTHON`
`[verified native\_wasb\_runner.py:89]` ? so no single venv/freeze can deliver the GPU-quality path. **The honest v1 is a LIGHT tier** (Gemini + motion + cpu-mock, **zero*** torch/GPU) that preserves CUJ-1.5/8; the native-WASB GPU path (CUJ-6/7) ships **owner-only** until two blockers clear (WASB-C3 weight provenance + a reproducible truly-local config). **LIGHT** is the **FIRST** shippable increment, not the terminal scope ? the sole completion goal (north star, L10) is the **FULL** quality path: native WASB + BYO training + in-app annotation. **Delivery is cross-platform** (Windows/macOS/Linux): the same local webserver powers the UI on all three, with **NO** native component** (L9). **Five small engine prerequisites must land before any packaging push** (\$7 P0). The single most actionable finding: **all mutable state is CWD-relative today and** **--setup/server disagree on config.json** ? a silent data-loss footgun that P0 must fix first.

902

903 ## 1. Problem & goal

904 **Goal:** a non-technical user (the primary persona: an India coach/parent on Windows ? `docs/UI\_PROPOSAL.md` §2) **downloads and runs** the indexer **without sourcing the repo**. The app spins up a **local web service** for (a) inference video?rallies?reel, (b) BYO **training**, and (c) **rally annotation** if needed ? runnable **locally and against cloud services** ? and **preserves every completed CUJ** (\$ CUJ map).

905 @@ -75,7 +75,7 @@ Build `khelsutra/web` (`npm ci && npm run build`) **at package time** and vendor

906 | Tier | Audience | Contents | Install UX |

907 |-----|-----|-----|-----|

908 | **LIGHT** (v1 first increment) | Non-tech India coach/parent (Win/Mac/Linux); any CPU box | wheel + bundled SPA + Gemini/motion/cpu-mock; **no torch/GPU/weights**; ffmpeg via per-OS offer; defaults truly-local; user-supplied Gemini key | **Cross-platform, no native component**: **uv tool install** / a generated install script + a **pure-Python launcher** (`indexer --app` ? boot server ? `webbrowser.open 127.0.0.1:8000`, free-port fallback). Optional thin per-OS shortcut (.bat/.command/.desktop) wrapping the same command. First run: cpu-mock self-test ? 3-step wizard. |

909 -| **HEAVY / local-GPU** (owner + opt-in power user) | Owner w/ golden corpus + CUDA box | **LIGHT** + micromamba WASB env (py3.8/torch1.11) via `\$WASB\_PYTHON`; main torch out-of-band via uv; weights SHA-256-pinned. **Gated on C3 + reproducible config**; ONNX supersedes this env | opt-in "Enable local GPU" step; resumable progress (multi-GB on Indian bandwidth); post-install `torch.cuda.is\_available()` validate; falls back to **LIGHT**. |

910 +| **HEAVY / local-GPU** (owner + opt-in power user) | Owner w/ golden corpus + CUDA box | **LIGHT** + micromamba WASB env (py3.10/torch2.6, #506) via `\$WASB\_PYTHON`; main torch out-of-band via uv; weights SHA-256-pinned. **Gated on C3 + reproducible config**; ONNX supersedes this env | opt-in "Enable local GPU" step; resumable progress (multi-GB on Indian bandwidth); post-install `torch.cuda.is\_available()` validate; falls back to **LIGHT**. |

911 | **CLOUD / server** | GCP deploy + BYO-GPU-VM | the `deploy/cloudrun/Dockerfile` image (micromamba-ified); cloud-serving + gcs; behind Cloudflare Access | `gcloud run jobs create` per `deploy/cloudrun/README.md`, or `deploy/gcp` VM scripts. Bind?`0.0.0.0` (config-gated) + edge-auth mandatory. |

912

913 ## 5. Threat model / risk table (local appliance)

914 diff --git a/tests/test_compute_backend.py b/tests/test_compute_backend.py

```

915     index a41ce7d..a49c03a 100644
916     --- a/tests/test_compute_backend.py
917     +++ b/tests/test_compute_backend.py
918     @@ -690,7 +690,14 @@ def test_dockerfile_has_required_directives():
919         assert "FROM nvidia/cuda" in text # GPU base (CUDA)
920         assert "ffmpeg" in text # stitch/decode
921         assert "fonts-dejavu" in text # the watermark filter font
922     - assert "torch==1.11.0+cu113" in text # the verified WASB stack
923     + # The #506 modernized WASB stack (L4-validated): torch 2.6 cu124 + PyNvVideoCodec,
and
924     + # the NVDEC/NVENC driver userspace libs PyNvVideoCodec dlopen (compute-only GPU
hosts
925     + # inject neither) ? a silent drop of any of these bricks
decode_backend=nvdec_resident.
926     + assert "torch==2.6.0" in text
927     + assert "download.pytorch.org/whl/cu124" in text
928     + assert "PyNvVideoCodec" in text
929     + assert "libnvcuvid.so.1" in text and "libnvidia-encode.so.1" in text
930     + assert "torch==1.11.0+cu113" not in text # the old stack is gone, not co-installed
931     assert "WASB-SBDT" in text # the shuttle detector repo
932     assert 'ENTRYPOINT ["python3", "-m", "backend.worker"]' in text
933     assert "WASB_PYTHON=/opt/conda/envs/wasb/bin/python" in text
934     diff --git a/tests/test_config_wasb.py b/tests/test_config_wasb.py
935     index 9d304df..887b9c2 100644
936     --- a/tests/test_config_wasb.py
937     +++ b/tests/test_config_wasb.py
938     @@ -102,6 +102,68 @@ def test_wasb_gpu_feed_knobs_reject_invalid_values():
939         AppConfig(indexing={"wasb": {"batch_size": -8}})
940
941
942     +def test_wasb_decode_backend_default_off_and_reaches_native_runner():
943     +    """#506: decode_backend defaults to "cpu" (the flag is default-OFF until the full
944     +    15-clip golden gate at the SERVED preset passes) and an explicit override reaches
945     +    the native runner's config."""
946     +    from backend.config.models import WasbIndexConfig
947     +    from backend.pipeline.detectors.native_wasb_runner import NativeWasbConfig
948     +
949     +    assert WasbIndexConfig().decode_backend == "cpu"
950     +    assert (
951     +        NativeWasbConfig.from_indexing_cfg(AppConfig().indexing).decode_backend
952     +        == "cpu"
953     +    )
954     +    idx = AppConfig(indexing={"wasb": {"decode_backend": "nvdec_resident"}}).indexing
955     +    assert NativeWasbConfig.from_indexing_cfg(idx).decode_backend == "nvdec_resident"
956     +
957     +
958     +def test_wasb_decode_backend_env_override_beats_config(monkeypatch):
959     +    from backend.pipeline.detectors.native_wasb_runner import NativeWasbConfig
960     +
961     +    monkeypatch.setenv("WASB_DECODE_BACKEND", "nvdec_resident")
962     +    cfg = NativeWasbConfig.from_indexing_cfg(AppConfig().indexing)
963     +    assert cfg.decode_backend == "nvdec_resident"
964     +
965     +
966     +def test_wasb_decode_backend_rejects_typos_and_non_cuda_device():
967     +    """A typo'd backend and the contradictory nvdec+cpu-device combo must fail at
968     +    config load, not deep inside a paid GPU run. device=auto stays allowed (the
969     +    native runner's healthcheck hard-fails if auto resolves to cpu)."""
970     +    import pytest
971     +
972     +    from backend.config.models import WasbIndexConfig
973     +
974     +    with pytest.raises(ValueError):
975     +        WasbIndexConfig(decode_backend="nvdec") # Literal rejects unknown backends
976     +    with pytest.raises(ValueError, match="requires device"):

```

```

977 +         WasbIndexConfig(decode_backend="nvdec_resident", device="cpu")
978 +     with pytest.raises(ValueError, match="requires device"):
979 +         AppConfig(
980 +             indexing={
981 +                 "wasb": {"decode_backend": "nvdec_resident", "device": "mps"}
982 +             }
983 +         )
984 +     # cuda (the default) and auto are both valid hosts for the GPU-resident path.
985 +     assert (
986 +         WasbIndexConfig(decode_backend="nvdec_resident").decode_backend
987 +         == "nvdec_resident"
988 +     )
989 +     assert (
990 +         WasbIndexConfig(decode_backend="nvdec_resident", device="auto").device
991 +         == "auto"
992 +     )
993 +
994 +
995 +def test_cloud_serving_preset_keeps_decode_backend_default_off():
996 +    """The rollout gate (#506): the cloud-serving preset must NOT opt into
997 +    nvdec_resident until the full golden set passes at the SERVED preset ? a
998 +    regression here would silently flip the default in production."""
999 +    from backend.config.presets import preset_for
1000 +
1001 +    assert "decode_backend" not in preset_for("cloud-serving")["indexing"]["wasb"]
1002 +
1003 +
1004 +def test_live_indexing_knobs_declared():
1005 +    fields = IndexingConfig.model_fields
1006 +    assert "yolo_prefetch_workers" in fields
1007 +diff --git a/tests/test_native_wasb_runner.py b/tests/test_native_wasb_runner.py
1008 +index b636300..9ae9208 100644
1009 +--- a/tests/test_native_wasb_runner.py
1010 ++++ b/tests/test_native_wasb_runner.py
1011 +@@ -458,6 +458,80 @@ def test_csv_name_matches_wsl_contract(tmp_path):
1012 +     assert os.path.basename(out) == f"{key}_wasb.csv"
1013 +
1014 +
1015 +    ## --- #506: GPU-resident NVDEC decode backend
1016 +    -----
1017 +def test_run_predict_omits_decode_backend_flag_by_default(tmp_path):
1018 +    """decode_backend defaults to "cpu" ? the flag is NOT passed, so the default argv
1019 +    (and its byte-parity guarantee) is unchanged by #506."""
1020 +    _, _out, _rm, argv, _cwd, _key = _drive_success(
1021 +        NativeWasbConfig(weights_path="/m/w.pth"), tmp_path
1022 +    )
1023 +    assert "--decode-backend" not in argv
1024 +
1025 +def test_run_predict_nvdec_resident_threads_flag_and_skips_cv2_paths(tmp_path):
1026 +    """nvdec_resident decodes on the GPU inside wasb_infer.py: the flag is threaded and
1027 +    NEITHER cv2 path flag (--stream-video / --frames_out_dir) rides along; there is no
1028 +    frame cache to clean."""
1029 +    _, out, rm, argv, _cwd, _key = _drive_success(
1030 +        NativeWasbConfig(weights_path="/m/w.pth", decode_backend="nvdec_resident"),
1031 +        tmp_path,
1032 +    )
1033 +    assert out is not None
1034 +    assert argv[argv.index("--decode-backend") + 1] == "nvdec_resident"
1035 +    assert "--stream-video" not in argv and "--frames_out_dir" not in argv
1036 +    assert argv[argv.index("--device") + 1] == "cuda"
1037 +    rm.assert_not_called()
1038 +
1039 +
1040 +def

```

```

test_run_predict_nvdec_resident_auto_without_gpu_aborts_before_subprocess(tmp_path):
1041 +     """device=auto resolving to cpu cannot host NVDEC decode: run_predict fails BEFORE
1042 +     launching the (paid) inference subprocess ? only the CUDA probe runs."""
1043 +     r = NativeWasbRunner(
1044 +         NativeWasbConfig(
1045 +             weights_path="/m/w.pth", device="auto", decode_backend="nvdec_resident"
1046 +         )
1047 +     )
1048 +     with patch.object(r, "_copy_infer_script", return_value=True), patch.object(
1049 +         r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda False")
1050 +     ) as spy:
1051 +         out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123")
1052 +         assert out is None
1053 +         assert spy.call_count == 1 # the probe only; wasb_infer.py never launched
1054 +
1055 +
1056 +def test_healthcheck_rejects_unknown_decode_backend(tmp_path):
1057 +     """A typo'd decode_backend fails fast with a clear message, before any
subprocess."""
1058 +     r = NativeWasbRunner(_ok_cfg(tmp_path, decode_backend="nvdec"))
1059 +     with patch.object(r, "_run") as spy:
1060 +         ok, msg = r.healthcheck()
1061 +         assert not ok and "decode_backend" in msg
1062 +         spy.assert_not_called()
1063 +
1064 +
1065 +def test_healthcheck_nvdec_resident_requires_effective_cuda(tmp_path):
1066 +     """auto?cpu fallback is fine for the cpu decode path but NOT for nvdec_resident:
1067 +     the healthcheck must hard-fail instead of warn-and-proceed."""
1068 +     r = NativeWasbRunner(
1069 +         _ok_cfg(tmp_path, device="auto", decode_backend="nvdec_resident")
1070 +     )
1071 +     with patch.object(
1072 +         r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda False")
1073 +     ):
1074 +         ok, msg = r.healthcheck()
1075 +         assert not ok and "nvdec_resident" in msg
1076 +
1077 +
1078 +def test_healthcheck_nvdec_resident_ok_with_cuda(tmp_path):
1079 +     r = NativeWasbRunner(
1080 +         _ok_cfg(tmp_path, device="cuda", decode_backend="nvdec_resident")
1081 +     )
1082 +     with patch.object(
1083 +         r, "_run", return_value=_proc(0, stdout="torch 2.6.0 cuda True")
1084 +     ):
1085 +         ok, _msg = r.healthcheck()
1086 +         assert ok
1087 +
1088 +
1089 +     # --- the detector_impl='native' seam (parallels the stub seam)
-----
1090 +     def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
1091 +         monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
1092

```

4. assistant (2026-07-07T04:01:53.522Z)

Now let me read the actual repo files. I'll start with the config models, the native runner, and the WSL runner.

5. user (2026-07-07T04:01:54.732Z)

```

1     """Pydantic models for the application configuration.
2

```

```

3     This module defines the canonical, validated, typed configuration for the
4     Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6     Unknown / typo'd config keys are rejected at load time via extra="forbid".
7     """
8
9     from __future__ import annotations
10
11     from typing import Any, List, Literal, Optional
12
13     from pydantic import BaseModel, Field, field_validator, model_validator
14
15     Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18     class ValidationRelevanceConfig(BaseModel):
19         court_green_lower: list[int] = Field(default=[35, 40, 40])
20         court_green_upper: list[int] = Field(default=[85, 255, 255])
21         court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24     class ValidationConfig(BaseModel):
25         min_resolution_width: int = 1280
26         min_resolution_height: int = 720
27         min_fps: float = 29.9
28         min_blur_threshold: float = 20.0
29         max_scene_cuts_per_minute: float = 0.5
30         pts_max_gap_seconds: float = Field(
31             default=10.0,
32             ge=0.0,
33             description=(
34                 "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
35                 "GoPro H.264/H.265 recordings typically have GOP intervals of 1?4s; "
36                 "a gap >10s suggests edited/missing content. Mobile phones, screen "
37                 "recordings, and other encoders may use different GOP sizes ? tune "
38                 "this threshold for your recording device."
39             ),
40         )
41         relevance: ValidationRelevanceConfig = Field(
42             default_factory=ValidationRelevanceConfig
43         )
44
45
46     class TrackNetConfig(BaseModel):
47         wsl_distro: str = "Ubuntu"
48         repo_dir: str = "~/models/TrackNetV4"
49         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
50         conda_env: str = "TrackNetV4"
51         weights_path: str = ""
52         queue_length: int = 5
53         stage_in_wsl: bool = True
54         wsl_stage_dir: str = "~/clips"
55         timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
56         velocity_threshold: float = 0.02
57         # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
58         # decoded frame cache) are deleted automatically ? they are pure intermediates,
59         # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
60         # true only for debugging. On FAILURE they are always kept so a re-run can resume.
61         keep_frames: bool = False
62         # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
63         the # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
64         not warns).
65         # 0 = no extra floor (the headroom guard still applies).
66         min_free_gb: float = 0.0

```

```

66
67
68 class WasbIndexConfig(BaseModel):
69     """Typed config for the live WASB shuttle substrate (indexing.wasb).
70
71     Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
72     these knobs actually take effect ? previously this whole block was silently dropped
73     because nested config sections defaulted to extra='ignore'.
74     """
75
76     wsl_distro: str = "Ubuntu"
77     repo_dir: str = "~/models/WASB-SBDT"
78     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
79     conda_env: str = "wasb"
80     weights_path: str = (
81         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
82     )
83     sport: str = "badminton"
84     wsl_stage_dir: str = "~/clips"
85     timeout_sec: int = 1800 # 30 min; kills a hung WSL/GPU call (0 = no timeout)
86     velocity_threshold: float = 0.02
87     # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner.
88     # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
89     # torch device for the native runner. "auto" = cuda-if-available-else-cpu (the M4
CPU fallback
90     # for a no-GPU box); "cuda" stays strict (hard-fails without a GPU ? no silent slow-
CPU downgrade).
91     device: str = "cuda" # auto | cuda | mps | cpu
92     python_bin: str = (
93         "python" # the `wasb` conda env python (invoked by path, no `conda activate`)
94     )
95     # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
96     # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates,
regenerable
97     # from the source. Set true only for debugging. On FAILURE they are kept so a re-run
resumes.
98     keep_frames: bool = False
99     # Absolute FLOOR (GB) on WSL free space (at wsl_stage_dir) before a run, on top of
the
100     # ~15x-source headroom the frame-stage guard always enforces (D11 ? it now BLOCKS,
not warns).
101     # 0 = no extra floor (the headroom guard still applies).
102     min_free_gb: float = 0.0
103     # FAST PATH ? decode the video on the fly and feed frames straight to the detector,
104     # skipping the slow per-frame PNG extraction that dominates a long clip (~46k PNGs
for a
105     # 12-min 1080p60 video, which overran the 30-min timeout). Output is bit-identical
to the
106     # PNG path (PNG is lossless) ? VERIFIED on a real GPU 2026-06-20 (native stream ==
WSL disk,
107     # 1194/1195 frames byte-identical; native run-to-run byte-identical /
deterministic).
108     # Tri-state:
109     # None = per-runner default ? the WSL runner keeps DISK extraction
(unchanged Windows
110     # behaviour); the Linux-native runner (GPU box) defaults to STREAMING
(the perf win).
111     # True/False = force on/off for BOTH runners (e.g. set False for a container cv2
can't seek).
112     stream_video: Optional[bool] = None
113     # GPU-feed tuning for the native runner (measured on the first real L4 serving run,
114     # 2026-07-03: the default batch 8 with single-process loading left the GPU <1%
utilized ?
115     # the pipeline was CPU-feed-bound). None = wasb_infer.py's parity defaults (batch 8,

```

```

no
116         # prefetch ? byte-identical to the historical path). The cloud-serving preset opts
into
117         # the measured fast values; local/WSL behaviour is unchanged unless set explicitly.
118         batch_size: Optional[int] = None
119         # Depth of the bounded producer-thread queue that overlaps CPU-side batch assembly
120         # (decode + PIL + ToTensor) with GPU inference in streaming mode. None/0 = off.
121         prefetch_batches: Optional[int] = None
122         # GPU-RESIDENT DECODE (#506) ? DEFAULT OFF. "nvdec_resident" moves
decode+resize+normalize
123         # onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
`grid_sample`
124         # resize replicating WASB's cv2.warpAffine EXACTLY (the #506 lesson: a naive
F.interpolate
125         # silently over-detects ? F1 0.58?0.07 ? and only shows on hard clips) ? GPU
ImageNet
126         # normalize ? HRNet. Validated on a real L4 (GX010128, SERVED preset, 2026-07-06):
~1.7x
127         # fps (35.8?60.2), GPU util 47?79% / CPU 62?35%, F1 0.574 ? baseline 0.582. Native
(Linux)
128         # runner only ? needs the torch-2.x + PyNvVideoCodec WASB env
(deploy/cloudrun/Dockerfile)
129         # with libnvcuvid/libnvidia-encode present, and device=cuda; the WSL runner ignores
it.
130         # Flip gate: the full 15-clip golden set at the SERVED preset (GX010128 must pass).
131         decode_backend: Literal["cpu", "nvdec_resident"] = "cpu"
132
133         @model_validator(mode="after")
134         def _nvdec_resident_requires_cuda(self) -> "WasbIndexConfig":
135             # NVDEC decode is CUDA-only; reject the contradictory combo at config load
instead of
136             # failing deep inside a paid GPU run. "auto" is allowed here ? the native
runner's
137             # healthcheck hard-fails if auto resolves to cpu with this backend on.
138             if self.decode_backend == "nvdec_resident" and self.device not in ("cuda",
"auto"):
139                 raise ValueError(
140                     f"indexing.wasb.decode_backend='nvdec_resident' requires device 'cuda'
or "
141                     f"'auto' (got {self.device!r}) ? NVDEC decode runs on the GPU."
142                 )
143             return self
144
145         @field_validator("batch_size")
146         @classmethod
147         def _batch_size_positive(cls, v: Optional[int]) -> Optional[int]:
148             # A DataLoader batch must be >=1; reject at config time so a typo can't reach
149             # `wasb_infer.py --batch-size` as an invalid value (the runner only passes the
flag
150             # when truthy, so 0 stays "off/parity-default").
151             if v is not None and v < 1:
152                 raise ValueError(f"indexing.wasb.batch_size must be >= 1 when set (got
{v})")
153             return v
154
155         @field_validator("prefetch_batches")
156         @classmethod
157         def _prefetch_batches_non_negative(cls, v: Optional[int]) -> Optional[int]:
158             # 0 = off (the default). Reject negatives: the runner passes any truthy value as
159             # `--prefetch-batches`, and `_PrefetchIter` clamps depth with max(1,
int(depth)), so a
160             # negative would SILENTLY enable prefetch at depth 1 instead of failing loudly.
161             if v is not None and v < 0:
162                 raise ValueError(
163                     f"indexing.wasb.prefetch_batches must be >= 0 when set (got {v})"

```



```

164         )
165         return v
166
167
168     class FusionConfig(BaseModel):
169         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
170 P2).
171
172         Every default reproduces control behaviour: the fusion segmenter persists the
173 shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']``
174 is
175 true ? the person-cue features, but it does NOT gate the served handoff unless a
176 threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court
177 mask
178 is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-
179 only);
180 supplied ? off-court persons (spectators / adjacent courts) are excluded.
181 """
182
183         # Which trajectory substrate seeds the windows (shuttle stays primary).
184         substrate: Literal["wasb", "tracknet"] = "wasb"
185         # Windowing velocity threshold for the fusion family (the engine reads
186         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
187         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
188         velocity_threshold: float = 0.02
189         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
190         # enabled ? so the default path never imports torch / touches the GPU.
191         cue_flags: dict[str, bool] = Field(default_factory=dict)
192         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
193         handoff.
194         # 0 = OFF (no gating; persisted candidates are always the full superset for offline
195         re-scoring).
196         min_crossings: int = Field(default=0, ge=0)
197         # Served Gate B: when the person cue is on, drop windows with median in-court
198         players
199         # < this. 0 = OFF.
200         min_players: int = Field(default=0, ge=0)
201         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
202         court_polygon: Optional[list[list[float]]] = None
203         # Net line for the person two-sided-motion split (person features only ? the shuttle
204         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
205         net_axis: Literal["x", "y"] = "y"
206         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
207
208
209     class IndexingConfig(BaseModel):
210         default_segmenter: str = "yolo_hybrid"
211         use_gpu: bool = True
212         # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
213         # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
214         # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
215         # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
216         # RALLY_DETECTOR_IMPL env var overrides this. See
217         pipeline/detectors/{stub,native_wasb}_runner.py.
218         detector_impl: str = "auto"
219         motion_threshold: float = 0.08
220         min_segment_duration: float = 3.0
221         dead_time_merge_gap: float = 2.0
222         chunk_duration: float = 90.0
223         chunk_overlap: float = 10.0
224         detector_model: str = "fasterrcnn_resnet50_fpn"
225         detector_score_threshold: float = 0.5
226         yolo_velocity_threshold: float = 0.15
227         yolo_frame_skip: int = 3
228         yolo_tracker: str = "bytetrack.yaml"

```

```

221     yolo_prefetch_workers: int = 2
222     min_action_window_duration: float = 2.0
223     # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0
= a window
224     # longer than this many seconds is recursively split at its largest internal
inactivity gap (?
225     # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't
swallow
226     # several rallies + the dead time between them. Never merges, only cuts (recall
can't drop).
227     # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip.
Measured n=13
228     # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0,
p=0.0005; cap=6
229     # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate
0.651?0.161). ***
230     max_window_duration: float = 6.0
231     window_min_split_gap: float = (
232         0.5 # min internal gap (s) that qualifies as a split point
233     )
234     # HARD-CAP fallback for W1: when True, an over-long window with NO internal
inactivity gap
235     # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob)
is
236     # chopped at its midpoint until ? max_window_duration, making the cap a true upper
bound.
237     # Only cuts (recall can't drop). OFF by default until measured/promoted.
238     window_hard_cap: bool = False
239     # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving
slowly
240     # (picked up / walked) above velocity_thresh, so the window over-extends its END
(the measured
241     # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose
trailing
242     # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only
cuts.
243     # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but
significant
244     # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13).
The W1 cap
245     # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md
"R1". ***
246     window_inactivity_end: float = 1.5
247     inactivity_rally_thresh: float = 0.20
248     inactivity_min_density: float = 0.30
249     # R4 density DENOMINATOR fix (code-review finding B). The R4 trim's fast-fraction
denominator is
250     # the COUNT of active samples in the trailing window, which sparse tracking inflates
(one fast
251     # sample after a gap ? 100% dense ? tail held open). When True, the denominator is
the window's
252     # TEMPORAL length instead. No-op under dense tracking (sample-count == temporal-
span), so it only
253     # corrects the sparse-track case. *** P2b PROMOTED DEFAULT-ON (2026-06-29): golden-
set A/B at the
254     # SERVED operating point (n=15, 595 rallies) ? tolF1 0.353?0.364 (?tolF1 +0.010,
sign 10W/4L, NO
255     # regression), |?end| 2.97?2.72s, seg_count_ratio ?0.103 (p=0.006), split_rate
?0.010 (p=0.016),
256     # R1 over-seg guard PASS; served_regresses(eps=0.0)=False ?
promote_if_served_safe(structural)=True.
257     # See docs/RALLY_DETECTION_FIXES_PLAN.md §7/§8. ***
258     inactivity_temporal_density: bool = True
259     # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active
blob: after

```

```

260         # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group
STILL longer
261         # than window_blob_factor × max_window_duration (no internal gap, no rally-end
signal ? e.g.
262         # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as
a FORCED cut
263         # (logged + flagged on the window) so the degenerate single-window outcome is
avoided and the
264         # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from
window_hard_cap
265         # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER
R4 and only
266         # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
267         window_blob_fallback: bool = False
268         # Magnitude gate that keeps R5 surgical: only a group longer than this ×
max_window_duration is a
269         # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual
is ~11×).
270         # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-
active
271         # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md
"R5".
272         window_blob_factor: float = 4.0
273         log_yolo_candidates: bool = True
274         store_yolo_candidates: bool = True
275         ai_handoff_padding: float = 2.0
276         ai_max_workers: int = 5
277         # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
278         # chunks of a high-bitrate (4K) source blow past the provider upload cap
279         # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
280         # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
281         # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
282         # Matches gemini_refine's --clip-scale-width default.
283         ai_chunk_scale_width: int = 1280
284         # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
285         # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff
entirely and
286         # emit their local candidate windows AS the rallies ? no provider, no API key,
nothing
287         # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the
Gemini
288         # path, or to operate with zero cloud dependency). The pure `gemini` segmenter
ignores this.
289         skip_ai_handoff: bool = False
290         # Optional debug knob: trim every video to the first N seconds before processing.
291         debug_duration: Optional[float] = None
292
293         tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
294         wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
295         fusion: FusionConfig = Field(default_factory=FusionConfig)
296
297         @field_validator("default_segmenter")
298         @classmethod
299         def segmenter_must_be_registered(cls, v: str) -> str:
300             # Lazy import: the segmenter registry imports modules that may import
301             # config, so we resolve it at validation time rather than at module load.
302             from backend.pipeline.segmenters.base import segmenter_registry
303
304             if segmenter_registry.get(v) is None:
305                 available = sorted(segmenter_registry.list_available().keys())
306                 raise ValueError(
307                     f"Segmenter '{v}' is not registered. Available: {' '.join(available)}"
308                 )
309             return v
310

```

```

311     @model_validator(mode="after")
312     def _chunk_step_must_be_positive(self) -> "IndexingConfig":
313         # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-
positive
314         # step makes `while t < duration: t += step` loop forever, growing memory before
any
315         # GPU work. Reject the bad config at load time instead.
316         if self.chunk_overlap >= self.chunk_duration:
317             raise ValueError(
318                 f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration
"
319                 f"({self.chunk_duration}); otherwise chunking never advances."
320             )
321         return self
322
323
324     class OutputConfig(BaseModel):
325         default_highlights_name: str = "highlights.mp4"
326         db_name: str = "sports_indexer.db"
327         # Directory where the DB, highlight reels, and processed clips are written.
328         # The CLI's --output-dir overrides this at startup (see backend/main.py:main).
329         output_dir: str = "output"
330
331
332     class WatermarkConfig(BaseModel):
333         """Branding overlay burned into each highlight clip during the per-clip re-encode
334         (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
335         (under `stitching.watermark` in config.json) to turn it off. The text is fully
336         configurable. The concat stage stays stream-copy (-c copy)."""
337
338         enabled: bool = True
339         text: str = "Generated by Khelsutra.guru"
340         logo_path: str = "" # optional transparent PNG overlay; "" = no logo
341         font_path: str = "" # optional .ttf; "" = use the fontconfig `font` family below
342         font: str = "Sans" # fontconfig family used when font_path is empty
343         font_size_ratio: float = Field(
344             default=0.035, gt=0.0, le=0.5
345         ) # text height as a fraction of frame height
346         opacity: float = Field(default=0.85, ge=0.0, le=1.0)
347         box: bool = True # faint backing box behind the text for legibility
348         margin: int = Field(default=24, ge=0) # px inset from the chosen corner
349         position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = (
350             "bottom-right"
351         )
352
353
354     class StitchingConfig(BaseModel):
355         begin_padding: float = 0.5
356         end_padding: float = 0.5
357         use_cache: bool = False
358         min_shot_count: int = 1
359         # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose
duration
360         # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
361         # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on
the
362         # fully-local no-AI path, whereas duration is derived from the rally's own
start/end. The
363         # local detector over-fires and its false positives are mostly SHORT (motion blips,
364         # background-court fragments), so this keeps the reel to the eye-catching long
rallies.
365         # OFF by default ? the detector output is unchanged, only which rallies reach the
reel.
366         min_rally_duration: float = Field(default=0.0, ge=0.0)
367         watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)

```

```

368
369
370 class LoggingConfig(BaseModel):
371     """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
372
373     # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
374     # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
375     # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
376     # them to WARNING; set true to restore full chatter for request-flow debugging.
377     verbose_http: bool = False
378
379
380 class SecurityConfig(BaseModel):
381     # When set, /api/process video_path must resolve under this directory (hosted/tenant
382     # mode). Default None preserves the single-user local 'any local file' workflow.
383     allowed_video_root: Optional[str] = None
384
385     # --- Chunked upload (B2, PR-2) ---
386     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
387     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
388     # contain upload_dir or /api/process will reject the uploaded paths.
389     upload_dir: str = "output/uploads"
390     # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
391     # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
392     max_upload_mb: int = 4096
393     max_part_mb: int = 95
394     allowed_upload_extensions: List[str] = ["mp4", "mov"]
395
396     # --- M9 edge auth (Cloudflare Access, D9) ---
397     # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
398     # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
399     # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
400     # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.
401     # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
402     # application's audience tag. NO secrets here ? these are public identifiers.
403     edge_auth_enabled: bool = False
404     cf_team_domain: str = (
405         "" # e.g. https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
406     )
407     cf_access_aud: str = "" # the CF Access application AUD tag the token must carry
408     # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
409     # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
410     # OFF ? Cf-Access is a header, not a cookie.)
411
412
413 class StorageConfig(BaseModel):
414     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + D0
Spaces
415     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
416     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
417     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
418     (boto3/google auth chains supply credentials). See ``backend/storage/``.
419
420
421     backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"

```

```

422         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
423         output_root: str = ""
424         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
425         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
426         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
427         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
428         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
429         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
430         # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
431         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
432         input_backend: Literal["local", "s3", "gcs", "gdrive", "gdrive-api"] = "local"
433         input_root: str = ""
434         # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
435         # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
436         # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
437         # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
438         # the legacy flat layout is used unchanged.
439         single_folder_per_video: bool = False
440         # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
441         workspace_root: str = ""
442         # Cloud namespace prefix under the root so per-video folders sit tidily beside
443         # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
444         workspace_prefix: str = "workspaces"
445         local: dict[str, Any] = Field(default_factory=dict)
446         s3: dict[str, Any] = Field(default_factory=dict)
447         gcs: dict[str, Any] = Field(default_factory=dict)
448         gdrive: dict[str, Any] = Field(default_factory=dict)
449         # gdrive-api settings (e.g. {"root_folder_id": "<id>"}); read via the
hyphen?underscore lookup in
450         # backend.storage.get_storage. NO secrets ? creds come from ADC over the Drive
scope.
451         gdrive_api: dict[str, Any] = Field(default_factory=dict)
452
453
454     class DeploymentConfig(BaseModel):
455         """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
456         knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
457
458         **Pure / additive:** the empty default profile (``""``) applies NO preset, so the
pipeline
459         behaves exactly as before this section existed. A profile is opt-in (``config.json``
``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
460         *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
461         these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
462         (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
463
464         Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
465
466

```

```

467     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
468     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
469     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
470     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
471     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
472     # Cloud-serving throughput cap (#466): how many DETECT jobs the control-plane drain
may dispatch +
473     # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound
Cloud Run Job
474     # execution (the GPU work is on ephemeral L4s, WASB-only has no Gemini), so ? unlike
a truly-local
475     # single box ? they need not serialize. Bounds concurrent PAID L4 executions. Non-
cloud_run targets
476     # ignore this and stay serial (max_workers=1), preserving the truly-local GPU/Gemini
serialization.
477     max_concurrent_remote_jobs: int = Field(default=4, ge=1, le=64)
478     # Poll budget for one dispatched remote job (CONTROL_PLANE_DURABLE_REBUILD $4.3).
Must sit
479     # ABOVE the worker Job's own --task-timeout (3600s) so the worker's timeout stays
the
480     # authority and the dispatcher's one-shot post-timeout marker rescue still applies ?
the old
481     # 1800s ExecutionPolicy default undercut real 1080p60 jobs (~22 min detect +
queue/cold-start,
482     # observed on the 2026-07-05 owner CUJ) and could abandon a slow-but-healthy paid
GPU run.
483     remote_poll_max_wait_sec: float = Field(default=3900.0, gt=0)
484
485
486     class BillingConfig(BaseModel):
487         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
488
489         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
490         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
491         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
492         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
493         inserts a real, calibrated version and points ``pricebook_version`` at it.
494
495         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
496         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
497         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
498
499         enabled: bool = False
500         pricebook_version: str = (
501             "placeholder-v0" # the seeded $0 version; owner flips to a real one
502         )
503         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore market)
504         margin: float = 0.0 # resale markup (?0); 0 = bill at cost
505
506
507     class FlywheelConfig(BaseModel):
508         """Data-flywheel harness (COMPUTE_DECOUPLED_SERVING M11, D12) ? the
consent?capture?eval?goldenize
509         loop: on a CONSENTED adult job, durably **capture** the source + rally output into
the per-video
510         workspace ? **shadow-eval** owned-vs-Gemini ? **propose promotion** to the golden
TRAINING set, so

```

```

511         the owned model climbs toward the D11 ship floor.
512
513         **DEFAULT-OFF + INERT:** ``enabled=False`` ? nothing is captured/evaluated (byte-
identical to
514         today). ? **Consent is FAKED-DENY until counsel + the consent schema land:** even
``enabled=True``
515         captures **nothing**, because ``flywheel.consent_attested`` currently returns
``False`` for every
516         job (the privacy guardrail ? D10: adults-only, explicit opt-in, derived-data + auto-
delete-row).
517         This is the PLUMBING only; the real consent gate (counsel-reviewed ToS/DPA ? consent
columns ?
518         wiring) + the owned-model + golden-label data needed to *activate* shadow-
eval/promotion are
519         owner/counsel/GPU scope. Pairs with M9-consent.""
520
521         enabled: bool = False
522         workspace_prefix: str = (
523             "flywheel" # per-video workspace prefix for captured artifacts
524         )
525         shadow_eval: bool = True # run the owned-vs-Gemini shadow comparison on capture
526         min_promote_n: int = 5 # promote_if_served_safe `min_n` (the served-safe gate)
527
528
529     class AppConfig(BaseModel):
530         """Root configuration object.
531
532         All runtime configuration should be accessed via an instance of this class
533         (or the loader) rather than raw dict lookups.
534         """
535
536         sport: Sport = "badminton"
537         ai_provider: str = "gemini"
538         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
539         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
540         ai_model: str = "gemini-flash-latest"
541
542         validation: ValidationConfig = Field(default_factory=ValidationConfig)
543         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
544         output: OutputConfig = Field(default_factory=OutputConfig)
545         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
546         security: SecurityConfig = Field(default_factory=SecurityConfig)
547         logging: LoggingConfig = Field(default_factory=LoggingConfig)
548         storage: StorageConfig = Field(default_factory=StorageConfig)
549         deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
550         billing: BillingConfig = Field(default_factory=BillingConfig)
551         flywheel: FlywheelConfig = Field(default_factory=FlywheelConfig)
552
553         # Reject unknown keys ? typed config migration is complete (D3).
554         model_config = {
555             "extra": "forbid",
556             "validate_assignment": True,
557         }
558
559
560     # Convenience type alias for callers that want to annotate
561     Config = AppConfig
562

```

6. user (2026-07-07T04:01:55.422Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2

```



```

3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10    * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11    ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13    * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14    (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15    strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16    * weights / repo / python / scratch come from the environment first
17    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18    ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20    It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21    name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22    ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23    the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24    (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26    Acceptance gate (parity, NOT strict byte-equality): the SAME clip through the WSL
runner
27    (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28    candidate windows within a small float tolerance (~1e-3 px). cuDNN is non-
deterministic
29    ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30    run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31    vs the cached WSL-disk CSV matched 1194/1195 frames (the lone diff a ~5e-5 px edge
32    artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33    (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34    not the numbers.
35    """
36
37    import logging
38    import os
39    import shutil
40    import subprocess
41    from dataclasses import dataclass
42    from typing import List, Optional, Tuple
43
44    from backend.config.models import IndexingConfig
45    from backend.pipeline.detectors.base import DetectorRunner
46    from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
47
48    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
49    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
50
51    logger = logging.getLogger(__name__)
52

```

```

53     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
54     # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
55     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
56
57     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
58     # so the "cuda True/False" string they parse can never drift apart.
59     _CUDA_PROBE_CODE = (
60         "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
61     )
62
63     # Decode backends wasb_infer.py understands (#506). Mirrored by its --decode-backend
choices;
64     # healthcheck rejects anything else up front so a typo can't reach argparse as rc=2 mid-
run.
65     _VALID_DECODE_BACKENDS = ("cpu", "nvdec_resident")
66
67
68     @dataclass
69     class NativeWasbConfig:
70         """Resolved settings for a Linux-native WASB run.
71
72         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
73         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
74         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
75         """
76
77         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
78         python_bin: str = (
79             "python" # the `wasb` conda env python (invoked by path, no activate)
80         )
81         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
82         sport: str = "badminton"
83         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-available-else-cpu)
84         scratch_dir: str = "" # frame-extraction scratch (env); "" = next to the output dir
85         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no timeout)
86         keep_frames: bool = (
87             False # retain the decoded frame cache after success (debug only)
88         )
89         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
90         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
91         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
92         stream_video: bool = True
93         # GPU-feed tuning (indexing.wasb.batch_size / prefetch_batches). 0 = omit the flag ?
94         # wasb_infer.py's parity defaults (batch 8, no prefetch). The cloud-serving preset
opts in.
95         batch_size: int = 0
96         prefetch_batches: int = 0
97         # GPU-resident NVDEC decode (#506, indexing.wasb.decode_backend /
WASB_DECODE_BACKEND).
98         # "cpu" (default) = omit the flag ? wasb_infer.py's historical cv2 decode paths,
argv
99         # byte-identical. "nvdec_resident" = decode+resize+normalize on-GPU (torch-2.x env
only).
100         decode_backend: str = "cpu"
101
102     @classmethod
103     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "NativeWasbConfig":
104         if isinstance(idx_cfg, dict):
105             idx_cfg = IndexingConfig(**idx_cfg)

```

```

106         w = idx_cfg.wasb
107         env = os.environ.get
108         # stream_video is tri-state in config: None/absent => native default (stream);
109         an
110         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
111         sv = w.stream_video
112         return cls(
113             repo_dir=env("WASB_REPO_DIR") or w.repo_dir,
114             python_bin=env("WASB_PYTHON") or w.python_bin,
115             weights_path=env("WASB_WEIGHTS") or w.weights_path or "",
116             sport=w.sport,
117             device=env("WASB_DEVICE") or w.device,
118             scratch_dir=env("WASB_SCRATCH")
119             or env("RALLY_SCRATCH")
120             or env("TMPDIR")
121             or "",
122             timeout_sec=w.timeout_sec,
123             keep_frames=w.keep_frames,
124             stream_video=(True if sv is None else bool(sv)),
125             batch_size=int(w.batch_size or 0),
126             prefetch_batches=int(w.prefetch_batches or 0),
127             decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
128         )
129
130     class NativeWasbRunner(DetectorRunner):
131         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
132         docstring."""
133         def __init__(self, cfg: NativeWasbConfig):
134             self.cfg = cfg
135             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
136             healthcheck;
137             # lazily probed by run_predict if healthcheck was skipped. None = not yet
138             probed.
139             self._cuda_available: Optional[bool] = None
140
141             # --- low-level native exec (the single place tests patch)
142             -----
143             def _run(
144                 self, argv: List[str], cwd: Optional[str] = None
145             ) -> subprocess.CompletedProcess:
146                 logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
147                 return subprocess.run(
148                     argv,
149                     cwd=cwd,
150                     capture_output=True,
151                     text=True,
152                     timeout=(self.cfg.timeout_sec or None),
153                 )
154
155             # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
156             cpu) -----
157             def _probe_cuda(self) -> bool:
158                 """Probe `torch.cuda.is_available()` in the wasb env's python (a subprocess).
159                 Any failure
160                 (missing python / timeout / import error) is treated as 'no CUDA'."""
161                 try:
162                     res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
163                     except (FileNotFoundError, subprocess.TimeoutExpired):
164                         return False
165                     return res.returncode == 0 and "cuda True" in (res.stdout or "")
166
167             def _cuda_is_available(self) -> bool:
168                 """CUDA availability, cached across healthcheck + run_predict so we probe at

```

```

most once."""
164         if self._cuda_available is None:
165             self._cuda_available = self._probe_cuda()
166         return self._cuda_available
167
168     def _effective_device(self) -> str:
169         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
170         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
171         if self.cfg.device == AUTO:
172             return DeviceContext(AUTO, self._cuda_is_available()).effective
173         return self.cfg.device
174
175     def _repo_src(self) -> str:
176         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
177
178     def _copy_infer_script(self) -> bool:
179         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
180         repo_src = self._repo_src()
181         try:
182             os.makedirs(repo_src, exist_ok=True)
183             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
184             return True
185         except OSError as e:
186             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
187         return False
188
189     def _rm_rf(self, path: Optional[str]) -> None:
190         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
191         if path:
192             shutil.rmtree(path, ignore_errors=True)
193
194     def healthcheck(self) -> Tuple[bool, str]:
195         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
196         if not self.cfg.weights_path:
197             return False, (
198                 "WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path)."
199             )
200         weights = os.path.expanduser(self.cfg.weights_path)
201         if not os.path.exists(weights):
202             return False, f"WASB weights not found at {weights}"
203         repo_src = self._repo_src()
204         if not os.path.isdir(repo_src):
205             return False, (
206                 f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
207                 f"(set WASB_REPO_DIR / indexing.wasb.repo_dir)."
208             )
209         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
210         # argparse rc=2 from wasb_infer.py at run time.
211         if self.cfg.device not in VALID_DEVICES:
212             return False, (
213                 f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)})."
214             )
215         if self.cfg.decode_backend not in _VALID_DECODE_BACKENDS:
216             return False, (
217                 f"unknown decode_backend {self.cfg.decode_backend!r} ? valid: "
218                 f"{'', '.join(_VALID_DECODE_BACKENDS)})."
219             )

```

```

220         try:
221             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
222         except FileNotFoundError:
223             return (
224                 False,
225                 f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON).",
226             )
227         except subprocess.TimeoutExpired:
228             return False, "native torch healthcheck timed out."
229         out = (res.stdout or "").strip()
230         if res.returncode != 0:
231             return (
232                 False,
233                 f"{self.cfg.python_bin}` torch import failed: {(res.stderr or
234 out).strip()}",
235             )
236         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
237 subprocess).
238         self._cuda_available = "cuda True" in out
239         ctx = DeviceContext(self.cfg.device, self._cuda_available)
240         if ctx.cuda_required_but_missing:
241             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
242 a silent
243             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
244             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
245         if ctx.fell_back:
246             logger.warning(
247                 "device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO CPU; WASB "
248                 "inference will be slow. Set indexing.wasb.device=cuda to require a
249 GPU.",
250                 out,
251             )
252         # NVDEC decode is CUDA-only: an auto?cpu fallback (or explicit cpu/mps, which
253 the
254 # config validator already rejects) cannot run the GPU-resident path ? fail loud
255 # instead of letting wasb_infer.py die mid-run on a paid box.
256         if self.cfg.decode_backend == "nvdec_resident" and ctx.effective != "cuda":
257             return False, (
258                 f"decode_backend=nvdec_resident requires CUDA, but the effective device
259 is "
260                 f"{ctx.effective!r} ({out}) ? NVDEC decode runs on the GPU."
261             )
262         return True, out
263
264 def run_predict(
265     self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
266 ) -> Optional[str]:
267     """Run WASB natively on a video. Returns the path to the trajectory CSV, or
268 None.
269
270     Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
271 WSL
272 runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
273 the
274 same downstream contract, so the parity comparison is apples-to-apples.
275 """
276     output_dir = os.path.abspath(output_win_dir)
277     os.makedirs(output_dir, exist_ok=True)
278     # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
279     norm = str(video_win_path).replace("\\", "/")
280     stem, _ext = os.path.splitext(os.path.basename(norm))
281     # Content-derived key: same filename + different bytes -> different key (no
282 cache reuse).
283     key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
284     out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

```

```

275
276     if not self._copy_infer_script():
277         logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
278         return None
279
280     weights = os.path.expanduser(self.cfg.weights_path)
281     argv = [
282         self.cfg.python_bin,
283         "wasb_infer.py",
284         "--video",
285         os.path.abspath(str(video_win_path)),
286     ]
287     frames_dir: Optional[str] = None
288     if self.cfg.decode_backend == "nvdec_resident":
289         # GPU-resident path (#506): wasb_infer.py decodes the video on the GPU
290         # (PyNvVideoCodec NVDEC) ? no PNG extraction and no cv2 streaming, so
291         # --stream-video nor --frames_out_dir applies and there is no frame cache to
292         # clean.
293         argv += ["--decode-backend", self.cfg.decode_backend]
294     elif self.cfg.stream_video:
295         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
296         # to disk.
297         argv.append("--stream-video")
298     else:
299         scratch = self.cfg.scratch_dir or output_dir
300         frames_dir = os.path.join(scratch, f"{key}_frames")
301         argv += ["--frames_out_dir", frames_dir]
302     # M4: resolve device="auto" to the effective device (cuda-if-available-else-
303     # cpu); an
304     # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
305     # wasb_infer.py.
306     device = self._effective_device()
307     if self.cfg.decode_backend == "nvdec_resident" and device != "cuda":
308         # Mirrors the healthcheck gate for callers that skipped it (e.g. auto?cpu on
309         # a
310         # GPU-less box): NVDEC decode cannot run without CUDA ? fail before the
311         # subprocess.
312         logger.error(
313             "decode_backend=nvdec_resident requires CUDA, but the effective device
314             is "
315             "%r ? aborting before the WASB subprocess.",
316             device,
317         )
318         return None
319     argv += [
320         "--weights",
321         weights,
322         "--sport",
323         self.cfg.sport,
324         "--device",
325         device,
326         "--out",
327         out_csv,
328     ]
329     # GPU-feed tuning: pass only explicit non-zero values so the default argv (and
330     # its
331     # byte-parity guarantee) is unchanged when the knobs are unset.
332     if self.cfg.batch_size:
333         argv += ["--batch-size", str(self.cfg.batch_size)]
334     if self.cfg.prefetch_batches:
335         argv += ["--prefetch-batches", str(self.cfg.prefetch_batches)]
336     logger.info(

```

```

330         "Running native WASB inference (device=%s%s) on %s ...",
331         device,
332         " [auto?fallback]" if self.cfg.device == AUTO and device != "cuda" else "",
333         os.path.basename(norm),
334     )
335     try:
336         res = self._run(argv, cwd=self._repo_src())
337     except FileNotFoundError:
338         logger.error(
339             "python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin
340         )
341         return None
342     except subprocess.TimeoutExpired:
343         logger.error(
344             "WASB native inference timed out after %ss.", self.cfg.timeout_sec
345         )
346         return None
347     if res.returncode != 0:
348         logger.error(
349             "WASB native inference failed:\n%s",
350             (res.stderr or res.stdout or "")[-2000:],
351         )
352         return None
353     if not os.path.exists(out_csv):
354         logger.error(
355             "WASB native finished but expected CSV not found at %s", out_csv
356         )
357         return None
358     logger.info("WASB trajectory CSV: %s", out_csv)
359
360     # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
361     # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
362     # (Streaming never materializes frames, so there is nothing to drop on that
path.)
363     if not self.cfg.keep_frames and frames_dir:
364         logger.info(
365             "Cache hygiene: removing frame cache for %s "
366             "(set indexing.wasb.keep_frames=true to retain).",
367             key,
368         )
369         self._rm_rf(frames_dir)
370     return out_csv
371
372     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
373     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
374     trajectory_to_action_windows = staticmethod(
375         TrackNetRunner.trajectory_to_action_windows
376     )
377

```

7. assistant (2026-07-07T04:03:31.310Z)

Now the wasb_infer.py file, the WSL runner, and the reference implementation.

8. user (2026-07-07T04:03:33.046Z)

```

1240         "--frames_out_dir", help="where to extract frames when --video is used"
1241     )
1242     ap.add_argument(
1243         "--stream-video",
1244         action="store_true",
1245         help="FAST PATH: decode --video on the fly and feed frames straight to "

```

```

1246         "the detector (no PNG extraction). Output-equivalent to the disk path.",
1247     )
1248     ap.add_argument(
1249         "--decode-backend",
1250         default="cpu",
1251         choices=list(DECODE_BACKENDS),
1252         help="cpu (default) = the historical cv2 decode paths (disk frames or "
1253         "--stream-video). nvdec_resident = GPU-RESIDENT decode+resize+normalize "
1254         "(#506): PyNvVideoCodec NVDEC + cv2-faithful affine grid_sample, CUDA-only, "
1255         "~1.7x on an L4. Requires --video; detections cache separately from cpu "
1256         "(trajectory-level, not byte-level, equivalence).",
1257     )
1258     ap.add_argument("--weights", required=True)
1259     ap.add_argument("--out", required=True)
1260     ap.add_argument("--sport", default="badminton")
1261     ap.add_argument(
1262         "--device",
1263         default="cuda",
1264         choices=["cuda", "mps", "cpu"],
1265         help="torch device for the detector (default cuda = the WSL parity path)",
1266     )
1267     ap.add_argument(
1268         "--limit", type=int, default=0, help="cap #frames (for quick tests)"
1269     )
1270     ap.add_argument("--batch-size", type=int, default=8)
1271     ap.add_argument("--num-workers", type=int, default=4)
1272     ap.add_argument(
1273         "--prefetch-batches",
1274         type=int,
1275         default=0,
1276         help="streaming mode: overlap CPU batch assembly with GPU inference via a "
1277         "bounded producer thread of this queue depth (0 = off, the parity default)",
1278     )
1279     ap.add_argument(
1280         "--log-every-batches", type=int, default=50, help="progress log cadence"
1281     )
1282     ap.add_argument(
1283         "--cache-dir",
1284         default=None,
1285         help="resumable detector cache dir (default: <out>_wasbcache next to --out)",
1286     )
1287     ap.add_argument(
1288         "--flush-every",
1289         type=int,
1290         default=1000,
1291         help="fsync the detector cache every N windows (bounds reboot loss)",
1292     )
1293     ap.add_argument(
1294         "--fresh", action="store_true", help="ignore any existing cache and start over"
1295     )
1296     args = ap.parse_args()
1297
1298     if args.decode_backend == "nvdec_resident":
1299         # GPU-resident branch FIRST: it decodes the video itself (NVDEC), so neither the
1300         # --stream-video cv2 path nor the extract-frames fallback below may run.
1301         if not args.video:
1302             raise SystemExit("--decode-backend nvdec_resident requires --video")
1303         if args.device != "cuda":
1304             raise SystemExit(
1305                 "--decode-backend nvdec_resident requires --device cuda "
1306                 "(NVDEC decode runs on the GPU)"
1307             )
1308         run_video_nvdec_resident(
1309             args.video,
1310             args.weights,

```



```

1311         args.out,
1312         args.sport,
1313         args.limit,
1314         batch_size=args.batch_size,
1315         log_every_batches=args.log_every_batches,
1316         cache_dir=args.cache_dir,
1317         flush_every=args.flush_every,
1318         fresh=args.fresh,
1319     )
1320     return
1321
1322     if args.stream_video:
1323         if not args.video:
1324             raise SystemExit("--stream-video requires --video")
1325         run_video_streaming(
1326             args.video,
1327             args.weights,
1328             args.out,
1329             args.sport,
1330             args.limit,
1331             batch_size=args.batch_size,
1332             log_every_batches=args.log_every_batches,
1333             cache_dir=args.cache_dir,
1334             flush_every=args.flush_every,
1335             fresh=args.fresh,
1336             device=args.device,
1337             prefetch_batches=args.prefetch_batches,
1338         )
1339     return
1340
1341     frames_dir = args.frames_dir
1342     if args.video:
1343         frames_dir = extract_frames(
1344             args.video, args.frames_out_dir or (args.video + "_frames")
1345         )
1346     if not frames_dir:
1347         raise SystemExit("provide --frames_dir or --video")
1348     run(
1349         frames_dir,
1350         args.weights,
1351         args.out,
1352         args.sport,
1353         args.limit,
1354         batch_size=args.batch_size,
1355         num_workers=args.num_workers,
1356         log_every_batches=args.log_every_batches,
1357         cache_dir=args.cache_dir,
1358         flush_every=args.flush_every,
1359         fresh=args.fresh,
1360         device=args.device,
1361     )
1362
1363
1364     if __name__ == "__main__":
1365         main()
1366

```

9. user (2026-07-07T04:03:34.387Z)

```

backend\pipeline\segmenters\base.py:56:         "no segments found" from "something went wrong"
(missing key, healthcheck
backend\pipeline\segmenters\tracknet_hybrid.py:9:The pipeline engine (healthcheck -> trajectory
-> windows -> AI handoff -> DB)
backend\pipeline\segmenters\trajectory_hybrid.py:7:     healthcheck gate -> P2 trajectory ->
candidate windows (+ persistence)

```

```

backend\pipeline\segmenters\trajectory_hybrid.py:183:      # an unreadable video, an
unavailable detector (native-not-supported), the env healthcheck
backend\pipeline\segmenters\trajectory_hybrid.py:282:      # --- Phase 1: env healthcheck
(fail fast with a clear message) ---
backend\pipeline\segmenters\trajectory_hybrid.py:283:      ok, msg = runner.healthcheck()
backend\pipeline\segmenters\wasb_hybrid.py:11:The pipeline engine (healthcheck -> trajectory ->
windows -> AI handoff -> DB)
backend\pipeline\detectors\device.py:9:healthcheck hard-failed on a box with no GPU, so a CPU-
only Linux box (cpu-serving / a
backend\pipeline\detectors\base.py:11:- Provide a single place to evolve the healthcheck / error
contract and thread
backend\pipeline\detectors\base.py:21:- healthcheck() -> tuple[bool, str]
backend\pipeline\detectors\base.py:31:2. Implement run_predict and healthcheck (return (bool,
str) tuple).
backend\pipeline\detectors\base.py:41:6. Add unit tests that exercise the contract (isinstance,
healthcheck shape, run
backend\pipeline\detectors\base.py:51:- healthcheck can just check "weights file exists and
torch importable + CUDA?".
backend\pipeline\detectors\base.py:57:The healthcheck message is already logged by the hybrids
on failure. The reporting
backend\pipeline\detectors\base.py:216:      def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\native_wasb_runner.py:57:# Single torch/cuda probe code, used by BOTH
healthcheck and the lazy device-resolution probe
backend\pipeline\detectors\native_wasb_runner.py:64:# healthcheck rejects anything else up front
so a typo can't reach argparse as rc=2 mid-run.
backend\pipeline\detectors\native_wasb_runner.py:97:      # GPU-resident NVDEC decode (#506,
indexing.wasb.decode_backend / WASB_DECODE_BACKEND).
backend\pipeline\detectors\native_wasb_runner.py:100:      decode_backend: str = "cpu"
backend\pipeline\detectors\native_wasb_runner.py:126:
decode_backend=env("WASB_DECODE_BACKEND") or w.decode_backend,
backend\pipeline\detectors\native_wasb_runner.py:135:      # Cached CUDA-availability probe
(used to resolve device="auto"). Set by healthcheck;
backend\pipeline\detectors\native_wasb_runner.py:136:      # lazily probed by run_predict if
healthcheck was skipped. None = not yet probed.
backend\pipeline\detectors\native_wasb_runner.py:163:      """"CUDA availability, cached across
healthcheck + run_predict so we probe at most once.""""
backend\pipeline\detectors\native_wasb_runner.py:194:      def healthcheck(self) -> Tuple[bool,
str]:
backend\pipeline\detectors\native_wasb_runner.py:215:      if self.cfg.decode_backend not in
_VALID_DECODE_BACKENDS:
backend\pipeline\detectors\native_wasb_runner.py:217:      f"unknown decode_backend
{self.cfg.decode_backend!r} ? valid: "
backend\pipeline\detectors\native_wasb_runner.py:228:      return False, "native torch
healthcheck timed out."
backend\pipeline\detectors\native_wasb_runner.py:251:      if self.cfg.decode_backend ==
"nvdec_resident" and ctx.effective != "cuda":
backend\pipeline\detectors\native_wasb_runner.py:253:      f"decode_backend=nvdec_resident requires CUDA, but the effective device is "
backend\pipeline\detectors\native_wasb_runner.py:288:      if self.cfg.decode_backend ==
"nvdec_resident":
backend\pipeline\detectors\native_wasb_runner.py:292:      argv += ["--decode-backend",
self.cfg.decode_backend]
backend\pipeline\detectors\native_wasb_runner.py:303:      if self.cfg.decode_backend ==
"nvdec_resident" and device != "cuda":
backend\pipeline\detectors\native_wasb_runner.py:304:      # Mirrors the healthcheck gate
for callers that skipped it (e.g. auto?cpu on a
backend\pipeline\detectors\native_wasb_runner.py:307:      "decode_backend=nvdec_resident requires CUDA, but the effective device is "
backend\pipeline\detectors\stub_runner.py:84:      def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\tracknet_runner.py:105:      def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\tracknet_runner.py:123:      return False, "WSL healthcheck
timed out."
backend\pipeline\detectors\wasb_infer.py:215:      decode_backend: str = "cpu",
backend\pipeline\detectors\wasb_infer.py:223:      ``decode_backend`` follows the same pattern
(#506): pre-existing caches (no field)

```

```

backend\pipeline\detectors\wasb_infer.py:244:         and manifest.get("decode_backend", "cpu")
== decode_backend
backend\pipeline\detectors\wasb_infer.py:615:         decode_backend="cpu", # run() IS the cpu
decode path (disk frames / cv2 stream)
backend\pipeline\detectors\wasb_infer.py:636:         "decode_backend": "cpu",
backend\pipeline\detectors\wasb_infer.py:1004:             f"decode_backend=nvdec_resident
needs PyNvVideoCodec with RGB output "
backend\pipeline\detectors\wasb_infer.py:1006:             f"/ gpu_setup.sh stage them). Fall
back to decode_backend=cpu."
backend\pipeline\detectors\wasb_infer.py:1069:     ``decode_backend`` and never shared with cpu-
decoded runs. Same resumable
backend\pipeline\detectors\wasb_infer.py:1081:         raise
SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
backend\pipeline\detectors\wasb_infer.py:1115:         decode_backend="nvdec_resident",
backend\pipeline\detectors\wasb_infer.py:1136:         "decode_backend": "nvdec_resident",
backend\pipeline\detectors\wasb_infer.py:1298:         if args.decode_backend == "nvdec_resident":
backend\pipeline\detectors\wasb_runner.py:85:     def healthcheck(self) -> Tuple[bool, str]:
backend\pipeline\detectors\wasb_runner.py:97:         return False, "WSL healthcheck timed
out."

```

10. user (2026-07-07T04:03:45.404Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12
13    import logging
14    import os
15    import shlex
16    import subprocess
17    from dataclasses import dataclass
18    from typing import Optional, Tuple
19
20    from backend.config.models import IndexingConfig
21    from backend.pipeline.detectors.base import (
22        DetectorRunner,
23        WslCommandMixin,
24        wsl_tilde_quote,
25    )
26
27    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
28    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
29    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, to_wsl_mnt_path
30
31    logger = logging.getLogger(__name__)
32
33    # Windows path to the inference wrapper we stage into WSL.
34    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
35
36
37    @dataclass
38    class WasbConfig:
39        """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
40
41        distro: str = "Ubuntu"

```

```

42     repo_dir: str = "~/models/WASB-SBDT"
43     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
44     conda_env: str = "wasb"
45     weights_path: str = (
46         "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
47     )
48     sport: str = "badminton"
49     wsl_stage_dir: str = "~/clips"
50     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
51     keep_frames: bool = (
52         False # retain staged video + frame cache after success (debug only)
53     )
54     min_free_gb: float = 0.0 # absolute floor (GB) atop the ~15x-source BLOCK guard (0
= off)
55     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
56     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
57     # extraction that dominates a long clip. Output is bit-identical to the PNG path
58     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
59     stream_video: bool = False
60
61     @classmethod
62     def from_indexing_cfg(cls, idx_cfg: "IndexingConfig") -> "WasbConfig":
63         if isinstance(idx_cfg, dict):
64             idx_cfg = IndexingConfig(**idx_cfg)
65         w = idx_cfg.wasb
66         return cls(
67             distro=w.wsl_distro,
68             repo_dir=w.repo_dir,
69             conda_sh=w.conda_sh,
70             conda_env=w.conda_env,
71             weights_path=w.weights_path,
72             sport=w.sport,
73             wsl_stage_dir=w.wsl_stage_dir,
74             timeout_sec=w.timeout_sec,
75             keep_frames=w.keep_frames,
76             min_free_gb=w.min_free_gb,
77             stream_video=w.stream_video if w.stream_video is not None else False,
78         )
79
80
81     class WasbRunner(WslCommandMixin, DetectorRunner):
82         def __init__(self, cfg: WasbConfig):
83             self.cfg = cfg
84
85         def healthcheck(self) -> Tuple[bool, str]:
86             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
87             cmd = (
88                 f"conda activate {self.cfg.conda_env} && "
89                 f"python -c \"import torch; print('torch', torch.__version__, "
90                 f"'cuda', torch.cuda.is_available())\""
91             )
92             try:
93                 res = self._wsl_bash(cmd)
94             except FileNotFoundError:
95                 return False, "`wsl` not found ? Windows + WSL2 required."
96             except subprocess.TimeoutExpired:
97                 return False, "WSL healthcheck timed out."
98             out = (res.stdout or "").strip()
99             if res.returncode != 0:
100                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
101             return True, out
102
103         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
104             src_mnt = to_wsl_mnt_path(video_win_path)

```

```

105         reason = self.wsl_source_unreadable_reason(src_mnt)
106         if reason:
107             logger.error("Cannot stage video into WSL: %s", reason)
108             return None
109         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
110         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
111         {wsl_tilde_quote(dest)} && echo OK"
112         try:
113             res = self._wsl_bash(cmd)
114         except subprocess.TimeoutExpired:
115             logger.error("Staging video into WSL timed out.")
116             return None
117         if res.returncode != 0 or "OK" not in (res.stdout or ""):
118             logger.error(
119                 "Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip()
120             )
121             return None
122         return dest
123
124     def _stage_infer_script(self) -> bool:
125         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
126         src_mnt = to_wsl_mnt_path(_INFER_WIN)
127         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo"
128         OK"
129         res = self._wsl_bash(cmd)
130         return res.returncode == 0 and "OK" in (res.stdout or "")
131
132     def run_predict(
133         self, video_win_path: str, output_win_dir: str, video_id: Optional[str] = None
134     ) -> Optional[str]:
135         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
136
137         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
138         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
139         can never reuse each other's staged frames, resumable cache, or CSV.
140
141         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
142         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
143         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
144         # the CSV landed inside WSL while Windows polled the repo-relative path.
145         output_win_dir = os.path.abspath(output_win_dir)
146         os.makedirs(output_win_dir, exist_ok=True)
147         # Normalize for cross-platform basename (tests use Windows paths on Linux).
148         video_win_path = str(video_win_path).replace("\\", "/")
149         base = os.path.basename(video_win_path)
150         stem, ext = os.path.splitext(base)
151         # Unique, content-derived key: same filename + different bytes -> different key.
152         key = f"{stem}_{video_id[:12]}" if video_id else stem
153         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
154         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
155
156         if not self._stage_infer_script():
157             logger.error("Could not stage wasb_infer.py into WSL.")
158             return None
159         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
160         if staged_video is None:
161             return None
162         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
163
164         # Disk guard (D11): a 4K clip can extract ~100 GB of PNG frames. BLOCK before
165         extraction if
166         # the WSL stage fs can't hold ~15x the source ? a mid-run disk-fill wastes a
167         long run and
168         # looks like a quality fail. Best-effort measuring (an undeterminable read never
169         blocks).

```

```

165         # Streaming never materializes the PNGs, so the guard only matters off the fast
path.
166         if not self.cfg.stream_video:
167             shortfall = self._wsl_frame_stage_shortfall(video_win_path)
168             if shortfall:
169                 logger.error("WASB inference aborted ? %s", shortfall)
170                 return None
171
172         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
173         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
174         if self.cfg.stream_video:
175             frame_args = "--stream-video "
176         else:
177             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
178         cmd = (
179             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
180             f"python wasb_infer.py "
181             f"--video {wsl_tilde_quote(staged_video)} "
182             f"{frame_args}"
183             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
184             f"--sport {shlex.quote(self.cfg.sport)} "
185             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
186         )
187         logger.info("Running WASB inference on %s ...", base)
188         try:
189             res = self._wsl_bash(cmd)
190         except subprocess.TimeoutExpired:
191             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
192             return None
193         if res.returncode != 0:
194             logger.error(
195                 "WASB inference failed:\n%s", (res.stderr or res.stdout)[-2000:]
196             )
197             return None
198         if not os.path.exists(expected_csv_win):
199             logger.error(
200                 "WASB finished but expected CSV not found at %s", expected_csv_win
201             )
202             return None
203         logger.info("WASB trajectory CSV: %s", expected_csv_win)
204
205         # Success ? the trajectory CSV is the durable output; the staged video copy and
206         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
207         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
208         # so the frame cache + detector cache remain for a resumable re-run.
209         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
210         if not self.cfg.keep_frames:
211             stale = (
212                 [staged_video] if self.cfg.stream_video else [frames_dir, staged_video]
213             )
214             logger.info(
215                 "Cache hygiene: removing staged video%s for %s "
216                 "(set indexing.wasb.keep_frames=true to retain).",
217                 "" if self.cfg.stream_video else " + frame cache",
218                 key,
219             )
220             self._wsl_rm_rf(stale)
221             return expected_csv_win
222
223         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
224         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
225         trajectory_to_action_windows = staticmethod(

```

```
226         TrackNetRunner.trajectory_to_action_windows
227     )
228
```

11. user (2026-07-07T04:03:58.358Z)

```
1     """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3     WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4     own building blocks (ImageDataset transform + detector + online tracker) to run on
5     any folder of extracted frames and emit a trajectory CSV in the exact shape the
6     indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8         Frame,Visibility,X,Y
9
10    It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11    imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12    (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14    Resumable across reboots
15    -----
16    The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17    the slow, expensive part; the CPU tracker pass that turns detections into a
18    trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19    resume mid-stream). We exploit that split:
20
21    * The detector pass writes its per-window raw outputs incrementally to
22      ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23      records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24      most ``--flush-every`` windows of GPU work instead of the whole pass.
25    * On restart we replay the cached windows, skip the DataLoader ahead to the first
26      un-cached window, and continue. Once every window is cached, the detector pass
27      is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28      the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29      not the GPU hour.
30
31    The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32    to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33    weights, sport, window size, frame count, device, or decode backend change.
34
35    GPU-resident decode (#506)
36    -----
37    ``--decode-backend nvdec_resident`` (default ``cpu``) moves decode + resize + normalize
38    onto the GPU with no host round-trip: PyNvVideoCodec NVDEC decode ? affine
39    ``grid_sample`` resize replicating WASB's ``cv2.warpAffine`` exactly ? GPU ImageNet
40    normalize ? detector. Validated on a real L4 (issue #506): ~1.7x the cv2 streaming
41    path with rally F1 preserved on the hard golden clip. CUDA-only; needs the torch-2.x +
42    PyNvVideoCodec env (the WSL torch-1.11 env keeps ``cpu``, its behaviour untouched).
43
44    Usage (in WSL, from ~/models/WASB-SBDT/src):
45        python wasb_infer.py --frames_dir /path/frames --weights
46        ../pretrained_weights/wasb_badminton_best.pth.tar \
47        --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
48        every 1000] [--fresh]
49
50    """
51
52    import argparse
53    import csv
54    import glob
55    import json
56    import logging
57    import os
58    import os.path as osp
59    from collections import defaultdict
60    from contextlib import contextmanager
```

```

58
59     logger = logging.getLogger(__name__)
60
61     CACHE_VERSION = 1
62     _DET_FILE = "det_raw.jsonl"
63     _MANIFEST_FILE = "manifest.json"
64
65     # Pixel-decode backends (#506). "cpu" = the historical cv2 paths (disk frames or
66     # --stream-video); "nvdec_resident" = GPU-resident NVDEC decode + affine grid_sample
67     # resize + normalize (torch 2.x + PyNvVideoCodec, CUDA-only).
68     DECODE_BACKENDS = ("cpu", "nvdec_resident")
69
70     # ImageNet statistics ? MUST match WASB's build_img_transforms Normalize (the GPU-
71     resident
72     # frontend normalizes on-device instead of through torchvision's transform).
73     IMAGENET_MEAN = (0.485, 0.456, 0.406)
74     IMAGENET_STD = (0.229, 0.224, 0.225)
75
76     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
77         """Compose the WASB eval config via Hydra, overriding model/weights/device.
78
79         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
80         preserved),
81         ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
82         ``runner.gpus=[0]``;
83         cpu/mps run with no GPU list so the detector places tensors on ``device``.
84         """
85         from hydra import compose, initialize
86
87         gpus = "[0]" if device == "cuda" else "[]" # single GPU on cuda; none for cpu/mps
88         with initialize(config_path="configs", version_base=None):
89             cfg = compose(
90                 config_name="eval",
91                 overrides=[
92                     f"dataset={sport}",
93                     "model=wasb",
94                     f"detector.model_path={weights}",
95                     f"runner.device={device}",
96                     f"runner.gpus={gpus}", # default config assumes multiple GPUs; pin to
97                     this box
98                 ],
99             )
100             return cfg
101
102     def _frame_id(path: str) -> int:
103         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
104         180)."""
105         stem = osp.splitext(osp.basename(path))[0]
106         digits = "".join(ch for ch in stem if ch.isdigit())
107         return int(digits) if digits else -1
108
109     # ----- #
110     # Runtime-compat shims for the torch-2.x / NumPy-2.x env (#506). Both are exact
111     # no-ops on the verified WSL torch-1.11 + numpy<1.24 stack.
112     # ----- #
113     def _ensure_np_inf() -> None:
114         """NumPy 2.0 removed the ``np.Inf`` alias, which WASB's OnlineTracker still reads.
115         Restore it before any tracker use; a no-op on NumPy 1.x."""
116         import numpy as np
117
118         if not hasattr(np, "Inf"):
119             np.Inf = np.inf # type: ignore[attr-defined]

```



```
118
119
120 @contextmanager
```

12. user (2026-07-07T04:03:58.712Z)

```
920 # GPU-resident decode backend (#506): NVDEC decode ? affine grid_sample resize ?
921 # normalize ? detector, all on the GPU (no host round-trip). Validated on a real
922 # L4: ~1.7x the cv2 streaming path with rally F1 preserved (0.574 ? baseline 0.582
923 # on GX010128 at the SERVED preset).
924 # ----- #
925 def build_affine_grid(trans_inv, inp_w: int, inp_h: int, orig_w: int, orig_h: int):
926     """`F.grid_sample` sampling grid replicating `cv2.warpAffine(img, trans,
927     (inp_w, inp_h), INTER_LINEAR)` ? THE #506 lesson lives here.
928
929     The GPU resize MUST be WASB's affine warp (center/scale geometry + cv2's
930     integer-coordinates-are-pixel-centers convention), NOT a plain resize:
931     `F.interpolate`'s half-pixel mapping shifts every sample by a constant
932     sub-pixel offset, which tips the sigmoid past `score_threshold` on marginal
933     frames and silently over-detects ? F1 0.58 ? 0.07 on the hard clip GX010128,
934     while the easy clip masked it at 98% agreement. Locked by a cv2-parity unit test.
935
936     `trans_inv` is WASB's INVERSE affine (`get_affine_transform(..., inv=1)`),
937     mapping output px ? source px; each source position is normalized to [-1, 1]
938     via `gx = (2*x + 1)/W - 1` (cv2's half-pixel convention). Pure torch+numpy ?
939     no WASB imports ? so the parity test runs on any CPU box.
940
941     Returns a `[1, inp_h, inp_w, 2]` float32 grid for `F.grid_sample(...,
942     mode='bilinear', align_corners=False, padding_mode='border')`.
943     """
944     import numpy as np
945     import torch
946
947     ys, xs = torch.meshgrid(
948         torch.arange(inp_h, dtype=torch.float32),
949         torch.arange(inp_w, dtype=torch.float32),
950         indexing="ij",
951     )
952     out_coords = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3] output px
953     mt = torch.from_numpy(np.ascontiguousarray(trans_inv, dtype=np.float32)) # [2,3]
954     src = out_coords @ mt.transpose(0, 1) # [H,W,2] source px
955     gx = (2.0 * src[..., 0] + 1.0) / orig_w - 1.0
956     gy = (2.0 * src[..., 1] + 1.0) / orig_h - 1.0
957     return torch.stack([gx, gy], dim=-1).unsqueeze(0) # [1,H,W,2]
958
959
960 def _wasb_inverse_affines(orig_h: int, orig_w: int, out_wh, out_scales):
961     """Per-scale inverse affinies exactly as WASB's `ImageDataset` builds
962     `trans_outputs_inv` (`get_transform(img, wh, inv=1)`: center = frame middle,
963     scale = max(h, w), output size halved per extra scale). Returns
964     `{scale: np.ndarray[2, 3] float64}` ? float64 like `cv2.getAffineTransform`,
965     matching what the DataLoader path collates for the postprocessor."""
966     import numpy as np
967     from utils.image import get_affine_transform
968
969     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
970     s = max(orig_h, orig_w) * 1.0
971     out = {}
972     out_w, out_h = int(out_wh[0]), int(out_wh[1])
973     for scale in out_scales:
974         out[scale] = get_affine_transform(c, s, 0, [out_w, out_h], inv=1)
975         out_w //= 2
976         out_h //= 2
977     return out
978
```

```

979
980 class NvdecResidentFrontend:
981     """On-GPU frame source for the ``nvdec_resident`` detector pass (#506).
982
983     PyNvVideoCodec ``SimpleDecoder(output_color_type=RGB)`` decodes on the GPU's
984     NVDEC engine; frames arrive as CUDA tensors via DLPack and never touch the host.
985     ``read_chw`` then applies the cv2-faithful affine warp (``build_affine_grid``)
986     and ImageNet-normalizes on-device, yielding the same ``[3, inp_h, inp_w]``
987     float32 tensor per frame that the ImageDataset path assembles on the CPU.
988     """
989
990     def __init__(self, video_path: str, inp_wh, gpu_id: int = 0):
991         import PyNvVideoCodec as nvc
992         import torch
993
994         self._torch = torch
995         self._device = torch.device("cuda", gpu_id)
996         inp_w, inp_h = int(inp_wh[0]), int(inp_wh[1])
997         try:
998             self._dec = nvc.SimpleDecoder(
999                 video_path, gpu_id=gpu_id, output_color_type=nvc.OutputColorType.RGB
1000             )
1001         except Exception as e:
1002             raise RuntimeError(
1003                 f"PyNvVideoCodec RGB SimpleDecoder failed for {video_path}: {e!r} ? "
1004                 f"decode_backend=nvdec_resident needs PyNvVideoCodec with RGB output "
1005                 f"and libnvcuvid/libnvidia-encode on the box (deploy/cloudrun/Dockerfile "
1006                 f"/ gpu_setup.sh stage them). Fall back to decode_backend=cpu."
1007             ) from e
1008         self.count = len(self._dec)
1009         probe = self._decode_rgb_hwc(0)
1010         self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
1011         trans_inp_inv = _wasb_inverse_affines(
1012             self.orig_h, self.orig_w, (inp_w, inp_h), [0]
1013         )[0]
1014         self._grid = build_affine_grid(
1015             trans_inp_inv, inp_w, inp_h, self.orig_w, self.orig_h
1016         ).to(self._device)
1017         self._mean = torch.tensor(IMAGENET_MEAN, device=self._device).view(3, 1, 1)
1018         self._std = torch.tensor(IMAGENET_STD, device=self._device).view(3, 1, 1)
1019
1020     def _decode_rgb_hwc(self, index: int):
1021         """Frame ``index`` as an on-GPU HWC RGB uint8 tensor (zero-copy via DLPack)."""
1022         torch = self._torch
1023         t = torch.from_dlpack(self._dec[index])
1024         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
1025             t = t.permute(1, 2, 0) # CHW layout variant ? HWC
1026         if t.dim() != 3 or t.shape[-1] != 3:
1027             raise RuntimeError(
1028                 f"unexpected NVDEC frame shape {tuple(t.shape)} for frame {index} ? "
1029                 f"expected 3-channel RGB (is output_color_type=RGB supported here?)"
1030             )
1031         return t
1032
1033     def read_chw(self, index: int):
1034         """Decoded frame ``index`` ? the normalized ``[3, inp_h, inp_w]`` float32 CUDA
1035         tensor the detector expects (warpAffine-faithful resize + ImageNet
1036         normalize)."""
1037         import torch.nn.functional as F
1038
1039         hwc = self._decode_rgb_hwc(index)
1040         chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
1041         warped = F.grid_sample(
1042             chw, self._grid, mode="bilinear", align_corners=False, padding_mode="border"

```

```

1042         ).squeeze(0)
1043         return (warped - self._mean) / self._std
1044
1045     def release(self):
1046         self._dec = None
1047
1048
1049     def run_video_nvdec_resident(
1050         video_path: str,
1051         weights: str,
1052         out_csv: str,
1053         sport: str = "badminton",
1054         limit: int = 0,
1055         batch_size: int = 8,
1056         log_every_batches: int = 50,
1057         cache_dir: "str | None" = None,
1058         flush_every: int = 1000,
1059         fresh: bool = False,
1060     ) -> str:
1061         """GPU-resident path (#506): NVDEC-decode ``video_path`` on the GPU and feed
1062         warped+normalized frames straight to the detector ? decode, resize, and
1063         normalize never touch the host. CUDA-only by construction.
1064
1065         NOT byte-identical to the ``cpu`` decode paths (NVDEC's YUV?RGB and the GPU
1066         warp differ from cv2 at float precision) ? equivalence is trajectory-level
1067         (the #506 unit gate: 96.3% visibility agreement, median 0.036 px) and
1068         rally-F1-level (the golden gate), so the detection cache is keyed by
1069         ``decode_backend`` and never shared with cpu-decoded runs. Same resumable
1070         cache + tracker tail as ``run()`` otherwise.
1071         """
1072         import time
1073
1074         import numpy as np
1075         import torch
1076
1077         from detectors import build_detector
1078
1079         _ensure_np_inf()
1080         if not torch.cuda.is_available():
1081             raise SystemExit("decode_backend=nvdec_resident requires a CUDA GPU")
1082
1083         cfg = _build_cfg(weights, sport, "cuda")
1084         frames_in = int(cfg["model"]["frames_in"])
1085         input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
1086         output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
1087         out_scales = list(cfg["model"]["out_scales"])
1088
1089         fe = NvdecResidentFrontend(video_path, input_wh)
1090         try:
1091             total = min(fe.count, limit) if limit else fe.count
1092             if total < frames_in:
1093                 raise SystemExit(f"need >= {frames_in} frames, video has {total}")
1094             windows_total = total - frames_in + 1
1095             logger.info(
1096                 f"nvdec_resident: {total} frames ({fe.orig_w}x{fe.orig_h}), "
1097                 f"{windows_total} windows"
1098             )
1099
1100             # ---- cache setup / resume decision (mirrors run()); keyed nvdec_resident) --- #
1101             if cache_dir is None:
1102                 cache_dir = default_cache_dir(out_csv)
1103                 os.makedirs(cache_dir, exist_ok=True)
1104                 det_jsonl, _ = _cache_paths(cache_dir)
1105                 cache_key = "video:" + osp.abspath(video_path)
1106                 manifest = None if fresh else read_manifest(cache_dir)

```

```

1107         resume = manifest_compatible(
1108             manifest or {},
1109             frames_dir=cache_key,
1110             weights=weights,
1111             sport=sport,
1112             frames_in=frames_in,
1113             frames_total=total,
1114             device="cuda",
1115             decode_backend="nvdec_resident",
1116         )
1117     if resume:
1118         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
1119         logger.info(
1120             f"resuming from cache: {windows_done}/{windows_total} windows already
detected"
1121         )
1122     else:
1123         if manifest is not None:
1124             logger.info("cache incompatible with this run -> starting fresh")
1125             reset_cache(cache_dir)
1126             windows_done, det_by_fid = 0, defaultdict(list)
1127
1128     base_manifest = {
1129         "version": CACHE_VERSION,
1130         "frames_dir": cache_key,
1131         "weights": weights,
1132         "sport": sport,
1133         "frames_in": frames_in,
1134         "frames_total": total,
1135         "device": "cuda",
1136         "decode_backend": "nvdec_resident",
1137         "windows_total": windows_total,
1138         "windows_done": windows_done,
1139     }
1140     write_manifest(cache_dir, base_manifest)
1141
1142     # ---- GPU-resident detector pass over the un-cached windows ----- #
1143     if windows_done < windows_total:
1144         with _torch_load_compat():
1145             detector = build_detector(cfg)
1146             # {scale: [1,2,3] float64 CPU tensor} ? the postprocessor .cpu().numpy()'s
1147             # these, so CPU float64 matches the DataLoader-collated path exactly.
1148             trans_t = {
1149                 s: torch.from_numpy(np.ascontiguousarray(m)).unsqueeze(0)
1150                 for s, m in _wasb_inverse_affines(
1151                     fe.orig_h, fe.orig_w, output_wh, out_scales
1152                 ).items()
1153             }
1154             t0 = time.time()
1155             done = windows_done
1156             win_idx = windows_done
1157             n_batches = 0
1158             log_every = max(1, log_every_batches)
1159             flush_n = max(1, flush_every)
1160             pending = []
1161             window = [] # sliding [(fid, chw)] of the last frames_in decoded frames
1162             batch_fids, batch_imgs = [], []
1163             logger.info(
1164                 f"detecting on {windows_total - windows_done} remaining windows "
1165                 f"({total {windows_total}, batch={batch_size}, source=nvdec-resident)..."
1166             )
1167             det_f = open(det_jsonl, "a")
1168             try:
1169                 with torch.no_grad():
1170                     # Window i needs frames [i, i+frames_in) ? start decoding at the

```

```

1171         # first un-cached window's first frame (NVDEC seeks past the
prefix).
1172     for idx in range(windows_done, total):
1173         window.append((idx, fe.read_chw(idx)))
1174         if len(window) > frames_in:
1175             window.pop(0)
1176         if len(window) < frames_in:
1177             continue
1178         batch_fids.append([fid for fid, _ in window])
1179         batch_imgs.append(
1180             torch.cat([chw for _, chw in window], dim=0)
1181         )
1182         if len(batch_imgs) < batch_size and idx < total - 1:
1183             continue
1184         nb = len(batch_imgs)
1185         imgs = torch.stack(batch_imgs, dim=0)
1186         # .repeat (not .expand): the postprocessor calls .numpy() on
these.
1187         affine_mats = {
1188             s: m.repeat(nb, 1, 1) for s, m in trans_t.items()
1189         }
1190         batch_results, _ = detector.run_tensor(imgs, affine_mats)
1191         for ib in range(nb):
1192             frames_payload = []
1193             for ie in sorted(batch_results[ib].keys()):
1194                 fid = batch_fids[ib][ie]
1195                 plain = [_det_plain(d) for d in batch_results[ib][ie]]
1196                 frames_payload.append([fid, plain])
1197                 det_by_fid[fid].extend(plain)
1198             pending.append({"w": win_idx, "f": frames_payload})
1199             win_idx += 1
1200         done += nb
1201         n_batches += 1
1202         batch_fids, batch_imgs = [], []
1203         if len(pending) >= flush_n or done >= windows_total:
1204             _append_windows(det_f, pending)
1205             pending = []
1206             base_manifest["windows_done"] = done
1207             write_manifest(cache_dir, base_manifest)
1208         if n_batches % log_every == 0 or done >= windows_total:
1209             el = time.time() - t0
1210             new_done = done - windows_done
1211             rate = new_done / el if el > 0 else 0.0
1212             eta = (windows_total - done) / rate if rate > 0 else 0.0
1213             logger.info(
1214                 f"{done}/{windows_total} "
1215                 f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
1216                 f"elapsed {el:.0f}s | ETA {eta:.0f}s"
1217             )
1218         if pending: # safety net; the done>=windows_total branch already
flushed
1219             _append_windows(det_f, pending)
1220             base_manifest["windows_done"] = done
1221             write_manifest(cache_dir, base_manifest)
1222         finally:
1223             det_f.close()
1224             logger.info(f"detection done in {time.time() - t0:.0f}s; running
tracker...")
1225     else:
1226         logger.info("detector cache already complete -> tracker-only re-run")
1227     finally:
1228         fe.release()
1229
1230     return _tracker_pass_and_write(cfg, det_by_fid, out_csv)

```

```

1231
1232
1233 def main():
1234     ap = argparse.ArgumentParser()
1235     ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
1236     ap.add_argument(
1237         "--video", help="video file; frames are extracted internally (in WSL)"
1238     )
1239     ap.add_argument(

```

13. assistant (2026-07-07T04:04:54.038Z)

Now the reference implementation and the untracked test file.

14. user (2026-07-07T04:04:54.911Z)

```

1      #!/usr/bin/env python3
2      """FULL WASB inference under torch 2.x: CPU cv2 vs GPU-resident PyNvVideoCodec front-
ends.
3      Writes ~/t2/cpu_traj.csv and ~/t2/gpu_traj.csv (Frame,Visibility,X,Y) + parity
summary."""
4      import csv, functools, os, os.path as osp, sys, time
5      from collections import defaultdict
6      import numpy as np
7      if not hasattr(np, "Inf"):  # NumPy 2.0 removed np.Inf; WASB's OnlineTracker uses it.
8          np.Inf = np.inf
9      print = functools.partial(print, flush=True)
10
11      T2 = osp.expanduser("~/t2")
12      REPO_SRC = osp.join(T2, "WASB-SBDT", "src")
13      WEIGHTS = osp.join(T2, "wasb.pth.tar")
14      VIDEO = osp.expanduser(os.environ.get("T2_VIDEO", osp.join(T2, "g1.mp4")))
15      BASELINE_CSV = osp.expanduser(os.environ.get("T2_BASELINE", osp.join(T2,
"g1_traj_baseline.csv")))
16      sys.path.insert(0, REPO_SRC)
17
18      FRAMES_IN = 3; FRAMES_OUT = 3
19      INP_W, INP_H = 512, 288; OUT_W, OUT_H = 512, 288; SCALE = 0
20      IMAGENET_MEAN = np.array([0.485, 0.456, 0.406], dtype=np.float32)
21      IMAGENET_STD = np.array([0.229, 0.224, 0.225], dtype=np.float32)
22      LIMIT = int(os.environ.get("T2_LIMIT", "0") or "0")
23
24      def build_cfg():
25          from omegaconf import OmegaConf
26          cfg = {
27              "model": {"name": "hrnet", "frames_in": FRAMES_IN, "frames_out": FRAMES_OUT,
28                  "inp_height": INP_H, "inp_width": INP_W, "out_height": OUT_H, "out_width":
OUT_W,
29                  "rgb_diff": False, "out_scales": [0],
30                  "MODEL": {"EXTRA": {"FINAL_CONV_KERNEL": 1, "PRETRAINED_LAYERS": ["*"],
31                      "STEM": {"INPLANES": 64, "STRIDES": [1, 1]},
32                      "STAGE1": {"NUM_MODULES": 1, "NUM_BRANCHES": 1, "BLOCK": "BOTTLENECK",
"NUM_BLOCKS": [1], "NUM_CHANNELS": [32], "FUSE_METHOD": "SUM"},
33                      "STAGE2": {"NUM_MODULES": 1, "NUM_BRANCHES": 2, "BLOCK": "BASIC",
"NUM_BLOCKS": [2, 2], "NUM_CHANNELS": [16, 32], "FUSE_METHOD": "SUM"},
34                      "STAGE3": {"NUM_MODULES": 1, "NUM_BRANCHES": 3, "BLOCK": "BASIC",
"NUM_BLOCKS": [2, 2, 2], "NUM_CHANNELS": [16, 32, 64], "FUSE_METHOD": "SUM"},
35                      "STAGE4": {"NUM_MODULES": 1, "NUM_BRANCHES": 4, "BLOCK": "BASIC",
"NUM_BLOCKS": [2, 2, 2, 2], "NUM_CHANNELS": [16, 32, 64, 128], "FUSE_METHOD": "SUM"},
36                      "DECONV": {"NUM_DECONVS": 0, "KERNEL_SIZE": [], "NUM_BASIC_BLOCKS": 2}},
"INIT_WEIGHTS": True}},
37                  "detector": {"name": "tracknetv2", "model_path": WEIGHTS, "step": 3,
38                      "postprocessor": {"name": "tracknetv2", "score_threshold": 0.5, "scales":
[0],
39                      "blob_det_method": "concomp", "use_hm_weight": True}},

```

```

40         "dataloader": {"heatmap": {"name": "binary_fixed_size", "sigmas": [2.5], "mags":
[1.0], "min_value": 0.6}},
41         "tracker": {"name": "online", "max_disp": 300},
42         "runner": {"device": "cuda", "gpus": [0]}}
43     return OmegaConf.create(cfg)
44
45     def build_transforms(orig_h, orig_w):
46         from utils.image import get_affine_transform
47         c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
48         s = max(orig_h, orig_w) * 1.0
49         trans_input = get_affine_transform(c, s, 0, [INP_W, INP_H], inv=0)
50         trans_out_inv = get_affine_transform(c, s, 0, [OUT_W, OUT_H], inv=1)
51         return trans_input.astype(np.float32), trans_out_inv.astype(np.float32)
52
53     class Detector:
54         def __init__(self, cfg, device):
55             import torch
56             from models.hrnet import HRNet
57             from detectors.postprocessor import TracknetV2Postprocessor
58             self.device = device
59             self.model = HRNet(cfg["model"]).to(device)
60             ckpt = torch.load(WEIGHTS, map_location=device, weights_only=False)
61             state = ckpt["model_state_dict"]
62             if any(k.startswith("module.") for k in state):
63                 state = {k[len("module.")]: v for k, v in state.items()}
64             missing, unexpected = self.model.load_state_dict(state, strict=False)
65             if missing or unexpected:
66                 print("[detector] load_state_dict: {} missing, {}
unexpected".format(len(missing), len(unexpected)))
67             self.model.eval()
68             self.postprocessor = TracknetV2Postprocessor(cfg)
69         def run_tensor(self, imgs, affine_mats):
70             import torch
71             with torch.no_grad():
72                 imgs = imgs.to(self.device, non_blocking=True)
73                 preds = self.model(imgs)
74                 pp = self.postprocessor.run(preds, affine_mats)
75                 results = {}
76                 for bid in sorted(pp.keys()):
77                     results[bid] = {}
78                     for eid in sorted(pp[bid].keys()):
79                         dets = []
80                         for sc in sorted(pp[bid][eid].keys()):
81                             scores = pp[bid][eid][sc]["scores"]; xys = pp[bid][eid][sc]["xys"]
82                             for xy, score in zip(xys, scores):
83                                 dets.append({"xy": xy, "score": score, "scale": sc})
84                         results[bid][eid] = dets
85             return results
86
87     def normalize_chw_np(chw_rgb_float):
88         out = np.empty_like(chw_rgb_float)
89         for c in range(3):
90             out[c] = (chw_rgb_float[c] - IMAGENET_MEAN[c]) / IMAGENET_STD[c]
91         return out
92
93     class CpuFrontend:
94         name = "cpu"
95         def __init__(self, video_path):
96             import cv2
97             self.cv2 = cv2
98             self.cap = cv2.VideoCapture(video_path)
99             if not self.cap.isOpened():
100                 raise RuntimeError("cannot open {}".format(video_path))
101             self.count = int(self.cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
102             self.orig_w = int(self.cap.get(cv2.CAP_PROP_FRAME_WIDTH) or 0)

```

```

103         self.orig_h = int(self.cap.get(cv2.CAP_PROP_FRAME_HEIGHT) or 0)
104         self.trans_input, self.trans_out_inv = build_transforms(self.orig_h,
self.orig_w)
105         self._pos = 0
106     def read_chw(self, index):
107         assert index == self._pos, "cpu decode forward-only ({} != {})".format(index,
self._pos)
108         ok, frame_bgr = self.cap.read()
109         if not ok:
110             raise RuntimeError("failed to decode frame {}".format(index))
111         self._pos += 1
112         rgb = self.cv2.cvtColor(frame_bgr, self.cv2.COLOR_BGR2RGB)
113         warped = self.cv2.warpAffine(rgb, self.trans_input, (INP_W, INP_H),
flags=self.cv2.INTER_LINEAR)
114         chw = warped.transpose(2, 0, 1).astype(np.float32) / 255.0
115         return normalize_chw_np(chw)
116     def to_tensor(self, chw_np):
117         import torch
118         return torch.from_numpy(np.ascontiguousarray(chw_np))
119     def stack_window(self, chw_list):
120         import torch
121         return torch.cat([self.to_tensor(c) for c in chw_list], dim=0)
122     def release(self):
123         self.cap.release()
124
125     class GpuFrontend:
126         name = "gpu"
127         def __init__(self, video_path):
128             import torch, PyNvVideoCodec as nvc
129             self.torch = torch; self.nvc = nvc; self.gpu = 0
130             dev = torch.device("cuda", self.gpu); self.device = dev
131             self.rgb_native = True
132             try:
133                 self.dec = nvc.SimpleDecoder(video_path, gpu_id=self.gpu,
output_color_type=nvc.OutputColorType.RGB)
134             except Exception as e:
135                 print("[gpu] SimpleDecoder RGB failed ({!r}); fallback default".format(e))
136                 self.rgb_native = False
137                 self.dec = nvc.SimpleDecoder(video_path)
138             self.count = len(self.dec)
139             probe = self._decode_hwc_uint8(0)
140             self.orig_h, self.orig_w = int(probe.shape[0]), int(probe.shape[1])
141             self.trans_input, self.trans_out_inv = build_transforms(self.orig_h,
self.orig_w)
142             self.mean = torch.tensor(IMAGENET_MEAN, device=dev).view(3, 1, 1)
143             self.std = torch.tensor(IMAGENET_STD, device=dev).view(3, 1, 1)
144             # Affine sampling grid replicating WASB's cv2.warpAffine(trans_input,
INTER_LINEAR)
145             # exactly (NOT a plain resize): for each output px map back to source px via the
146             # inverse affine (trans_out_inv), normalized to [-1,1] with cv2's half-pixel
convention.
147             ys, xs = torch.meshgrid(torch.arange(INP_H, dtype=torch.float32),
148                                     torch.arange(INP_W, dtype=torch.float32), indexing="ij")
149             oc = torch.stack([xs, ys, torch.ones_like(xs)], dim=-1) # [H,W,3]
output px
150             Mt = torch.tensor(self.trans_out_inv, dtype=torch.float32) # [2,3]
out->orig
151             src = oc @ Mt.transpose(0, 1) # [H,W,2] orig
px
152             gx = (2.0 * src[..., 0] + 1.0) / self.orig_w - 1.0
153             gy = (2.0 * src[..., 1] + 1.0) / self.orig_h - 1.0
154             self.grid = torch.stack([gx, gy], dim=-1).unsqueeze(0).to(dev) # [1,H,W,2]
155     def _decode_hwc_uint8(self, index):
156         torch = self.torch
157         frame = self.dec[index]

```



```

158         t = torch.from_dlpack(frame)
159         if t.dim() == 3 and t.shape[0] == 3 and t.shape[-1] != 3:
160             t = t.permute(1, 2, 0).contiguous()
161         if not self.rgb_native:
162             if t.dim() == 3 and t.shape[-1] == 3:
163                 t = t[..., [2, 1, 0]].contiguous()
164             else:
165                 raise RuntimeError("gpu fallback non-3ch frame {}; need
NV12->RGB".format(tuple(t.shape)))
166         return t
167     def read_chw(self, index):
168         import torch.nn.functional as F
169         hwc = self._decode_hwc_uint8(index)
170         chw = hwc.permute(2, 0, 1).unsqueeze(0).float() / 255.0
171         resized = F.grid_sample(chw, self.grid, mode="bilinear",
172                                 align_corners=False, padding_mode="border").squeeze(0)
# affine-warp = cv2.matchTemplate(resized, self.mean) / self.std
173         return (resized - self.mean) / self.std
174     def stack_window(self, chw_list):
175         return self.torch.cat(chw_list, dim=0)
176     def release(self):
177         self.dec = None
178
179     def run(frontend_cls, out_csv, cfg, device):
180         import torch
181         from trackers.online import OnlineTracker
182         fe = frontend_cls(VIDEO)
183         total = fe.count if not LIMIT else min(fe.count, LIMIT)
184         print("{} frames={} (processing {}), orig={}x{}".format(fe.name, fe.count, total,
fe.orig_w, fe.orig_h))
185         if total < FRAMES_IN:
186             raise RuntimeError("need >= {} frames, have {}".format(FRAMES_IN, total))
187         detector = Detector(cfg, device)
188         trans_np = fe.trans_out_inv
189         is_gpu = fe.name == "gpu"
190         trans_dev = device if is_gpu else torch.device("cpu")
191         trans_1 = torch.from_numpy(np.ascontiguousarray(trans_np)).to(trans_dev)
192         tracker = OnlineTracker(cfg); tracker.refresh()
193         det_by_fid = defaultdict(list); window = []; t0 = time.time(); n_windows = 0
194         for idx in range(total):
195             chw = fe.read_chw(idx)
196             window.append((idx, chw))
197             if len(window) < FRAMES_IN:
198                 continue
199             if len(window) > FRAMES_IN:
200                 window.pop(0)
201             fids = [w[0] for w in window]
202             imgs = fe.stack_window([w[1] for w in window]).unsqueeze(0)
203             affine_mats = {SCALE: trans_1.unsqueeze(0)}
204             results = detector.run_tensor(imgs, affine_mats)
205             for eid in sorted(results[0].keys()):
206                 fid = fids[eid]
207                 det_by_fid[fid].extend(results[0][eid])
208             n_windows += 1
209             if n_windows % 500 == 0:
210                 el = time.time() - t0; fps = n_windows / el if el > 0 else 0.0
211                 print("{} windows={}/{} | {:.1f} win/s | {:.0f}s".format(fe.name,
n_windows, total - FRAMES_IN + 1, fps, el))
212             fe.release()
213             el = time.time() - t0; proc_fps = total / el if el > 0 else 0.0
214             print("{} detection done: {} frames, {} windows in {:.0f}s ({:.1f}
frames/s)".format(fe.name, total, n_windows, el, proc_fps))
215             rows = []
216             for fid in sorted(det_by_fid.keys()):
217                 frame_dets = [{"xy": np.asarray(d["xy"], dtype=float), "score":

```

```

float(d["score"]), "scale": int(d["scale"])} for d in det_by_fid[fid]]
218         r = tracker.update(frame_dets)
219         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
220         _write_csv(out_csv, rows)
221         visible = sum(1 for _, v, *_ in rows if v)
222         print("{} wrote {} rows -> {} (visible {}/{}).".format(fe.name, len(rows), out_csv,
visible, len(rows)))
223         return out_csv
224
225     def _write_csv(out_csv, rows):
226         tmp = out_csv + ".tmp"
227         with open(tmp, "w", newline="") as f:
228             w = csv.writer(f); w.writerow(["Frame", "Visibility", "X", "Y"])
229             for fid, vis, x, y in rows:
230                 w.writerow([fid, vis, x, y])
231         os.replace(tmp, out_csv)
232
233     def _load_traj(path):
234         out = {}
235         if not osp.exists(path):
236             return out
237         with open(path, newline="") as f:
238             for row in csv.DictReader(f):
239                 try:
240                     out[int(float(row["Frame"]))] = (int(float(row["Visibility"])),
float(row["X"]), float(row["Y"]))
241                 except (KeyError, ValueError):
242                     continue
243         return out
244
245     def _compare(na, a, nb, b):
246         if not a or not b:
247             print(" {} vs {}: empty (a={}, b={})".format(na, nb, len(a), len(b))); return
248         common = sorted(set(a) & set(b)); dists = []; agree = 0
249         for fid in common:
250             va, xa, ya = a[fid]; vb, xb, yb = b[fid]
251             if va == vb: agree += 1
252             if va == 1 and vb == 1: dists.append(float(np.hypot(xa - xb, ya - yb)))
253         vr = agree / len(common) if common else 0.0
254         if dists:
255             d = np.array(dists)
256             print(" {} vs {}: common={} both-vis={} | median={:.3f}px p95={:.3f}px | vis-
agree={:.2f}%".format(
257                 na, nb, len(common), len(dists), float(np.median(d)), float(np.percentile(d,
95)), vr * 100))
258         else:
259             print(" {} vs {}: common={} both-vis=0 | vis-agree={:.2f}%".format(na, nb,
len(common), vr * 100))
260
261     def parity_summary(cpu_csv, gpu_csv):
262         print("\n==== PARITY SUMMARY =====")
263         cpu = _load_traj(cpu_csv); gpu = _load_traj(gpu_csv); base =
_load_traj(BASELINE_CSV)
264         print("loaded: cpu={} gpu={} baseline={}".format(len(cpu), len(gpu), len(base)))
265         _compare("cpu", cpu, "gpu", gpu); _compare("cpu", cpu, "baseline", base);
_compare("gpu", gpu, "baseline", base)
266         print("=====\n")
267
268     def main():
269         import torch
270         print("torch {} | cuda={}".format(torch.__version__, torch.cuda.is_available()))
271         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
272         cfg = build_cfg()
273         pref = os.environ.get("T2_OUTPREFIX", "")
274         cpu_csv = osp.join(T2, pref + "cpu_traj.csv"); gpu_csv = osp.join(T2, pref +

```

```

"gpu_traj.csv")
275     only = os.environ.get("T2_ONLY", "")
276     fes = [(CpuFrontend, cpu_csv), (GpuFrontend, gpu_csv)]
277     if only == "gpu":
278         fes = [(GpuFrontend, gpu_csv)]
279     elif only == "cpu":
280         fes = [(CpuFrontend, cpu_csv)]
281     for fe_cls, out in fes:
282         try:
283             print("\n----- FRONT-END: {} -----".format(fe_cls.name))
284             run(fe_cls, out, cfg, device)
285         except Exception as e:
286             import traceback
287             print("[{}] FAILED: {!r}".format(fe_cls.name, e)); traceback.print_exc()
288     parity_summary(cpu_csv, gpu_csv)
289     print("done.")
290
291 if __name__ == "__main__":
292     main()
293

```

15. user (2026-07-07T04:05:29.003Z)

```

120     @contextmanager
121     def _torch_load_compat():
122         """Scope ``torch.load`` to ``weights_only=False`` while WASB builds its detector.
123
124         WASB's ``detector.py`` calls ``torch.load(model_path)`` with no ``weights_only``
125         argument; torch 2.6 flipped that default to True, which rejects the (pre-2.6,
126         owner-staged, trusted) WASB checkpoint. The kwarg is injected only when the
127         installed torch has it AND the caller didn't pass it, so torch 1.11 (no such
128         kwarg) and explicit callers are untouched."""
129         import inspect
130
131         import torch
132
133         real_load = torch.load
134         if "weights_only" not in inspect.signature(real_load).parameters:
135             yield # torch < 1.13 ? nothing to shim
136             return
137
138         def _load(*args, **kwargs):
139             kwargs.setdefault("weights_only", False)
140             return real_load(*args, **kwargs)
141
142         torch.load = _load
143         try:
144             yield
145         finally:
146             torch.load = real_load
147
148         # ----- #
149         # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
150         # ----- #
151
152     def _cache_paths(cache_dir: str):
153         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
154
155
156     def default_cache_dir(out_csv: str) -> str:
157         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
158         d = osp.dirname(osp.abspath(out_csv))
159         stem = osp.splitext(osp.basename(out_csv))[0]
160         return osp.join(d, stem + "_wasbcache")
161

```

```

162
163 def _atomic_write_text(path: str, text: str) -> None:
164     """Write then rename so a crash mid-write can't leave a half-written file."""
165     tmp = path + ".tmp"
166     with open(tmp, "w") as f:
167         f.write(text)
168         f.flush()
169         os.fsync(f.fileno())
170     os.replace(tmp, path)
171
172
173 def _det_plain(det) -> list:
174     """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
175     list."""
176     xy = det["xy"]
177     return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
178
179 def _dets_to_tracker(plain_list):
180     """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
181     array,
182     the tracker does vector math / np.linalg.norm on it)."""
183     import numpy as np
184     return [
185         {"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
186         for p in plain_list
187     ]
188
189
190 def write_manifest(cache_dir: str, manifest: dict) -> None:
191     _, mpath = _cache_paths(cache_dir)
192     _atomic_write_text(mpath, json.dumps(manifest, indent=2))
193
194
195 def read_manifest(cache_dir: str):
196     _, mpath = _cache_paths(cache_dir)
197     if not osp.exists(mpath):
198         return None
199     try:
200         with open(mpath) as f:
201             return json.load(f)
202     except (json.JSONDecodeError, OSError):
203         return None
204
205
206 def manifest_compatible(
207     manifest: dict,
208     *,
209     frames_dir: str,
210     weights: str,
211     sport: str,
212     frames_in: int,
213     frames_total: int,
214     device: str = "cuda",
215     decode_backend: str = "cpu",
216 ) -> bool:
217     """A cache is reusable only if the run that produced it matches this run's inputs.
218
219     ``device`` defaults to ``cuda`` so caches written before device-keying (no
220     ``device`` field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda
221     cache
222     (detections can differ numerically across devices), and vice-versa.

```

```

223     ``decode_backend`` follows the same pattern (#506): pre-existing caches (no field)
224     are ``cpu``-decoded, so they stay compatible with cpu runs ? but an
225     ``nvdec_resident``
226     run never reuses a cpu cache (NVDEC decode + the GPU affine warp are trajectory-
level,
227     not byte-level, equivalents), and vice-versa.
228     ``frames_dir`` is the **already-canonical cache key** the caller stored in the
manifest
229     (``run()`` passes its ``cache_key`` here, and writes the same value into the
manifest):
230     ``osp.abspath(frames_dir)`` for the frames-on-disk path, or the synthetic
``video:<abspath>``
231     key for the streaming path. We therefore compare it VERBATIM ? do NOT re-``abspath``
it here:
232     ``osp.abspath("video:/x.mp4")`` is not recognised as absolute and gets the CWD
prepended,
233     so a streaming cache would never match itself and every re-run would recompute
(issue #348).
234     """
235     if not manifest or manifest.get("version") != CACHE_VERSION:
236         return False
237     return (
238         manifest.get("frames_dir") == frames_dir
239         and manifest.get("weights") == weights
240         and manifest.get("sport") == sport
241         and manifest.get("frames_in") == frames_in
242         and manifest.get("frames_total") == frames_total
243         and manifest.get("device", "cuda") == device
244         and manifest.get("decode_backend", "cpu") == decode_backend
245     )
246
247
248 def reset_cache(cache_dir: str) -> None:
249     """Drop any existing cache so the next run starts clean."""
250     det_jsonl, mpath = _cache_paths(cache_dir)
251     for p in (det_jsonl, mpath):
252         if osp.exists(p):
253             os.remove(p)
254
255
256 def load_and_clean_cache(cache_dir: str):
257     """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
258     rebuild the per-frame detection accumulation, and rewrite the file to exactly that
259     prefix so a trailing half-written line from a crash can't corrupt future appends.
260
261     Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
262     [x, y, score, scale] candidates (accumulated across every window the frame is in,
263     in window order ? identical to a non-resumed run).
264     """
265     det_jsonl, _ = _cache_paths(cache_dir)
266     det_by_fid = defaultdict(list)
267     valid: list = []
268     if osp.exists(det_jsonl):
269         with open(det_jsonl, "r") as f:
270             for line in f:
271                 s = line.strip()
272                 if not s:
273                     continue
274                 try:
275                     obj = json.loads(s)
276                 except json.JSONDecodeError:
277                     break # truncated trailing line (crash mid-flush)
278                 if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
279                     break

```

```

280         valid.append(s)
281         for fid, plain in obj["f"]:
282             det_by_fid[int(fid)].extend([list(p) for p in plain])
283         # Guarantee a clean append boundary by rewriting only the good prefix.
284         _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
285     return len(valid), det_by_fid
286
287
288 def _append_windows(fh, objs) -> None:
289     """Append window records as JSONL and durably flush (bounded loss on crash)."""
290     for o in objs:
291         fh.write(json.dumps(o, separators=(",", ":")) + "\n")
292     fh.flush()
293     os.fsync(fh.fileno())
294
295
296 def _atomic_write_trajectory(out_csv: str, rows) -> None:
297     os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
298     tmp = out_csv + ".tmp"
299     with open(tmp, "w", newline="") as f:
300         w = csv.writer(f)
301         w.writerow(["Frame", "Visibility", "X", "Y"])
302         w.writerows(rows)
303         f.flush()
304         os.fsync(f.fileno())
305     os.replace(tmp, out_csv)
306
307
308 # ----- #
309 # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
310 # ----- #
311 _STREAM_FRAME_FMT = "{:08d}.png"
312
313
314 def synthetic_frame_path(index: int) -> str:
315     """Stable, index-encoding frame name used by the streaming path.
316
317     It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
318     downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
319     frames came from disk or from the in-memory stream. The streaming image loader
320     inverts this name back to an index to fetch the decoded frame.
321     """
322     return _STREAM_FRAME_FMT.format(int(index))
323
324
325 def build_windows(frames: list, frames_in: int):
326     """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
327     + checkpoint). Pure list math, identical for the disk and streaming paths.
328
329     Returns a list of windows, each a list of ``frames_in`` consecutive paths
330     (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
331     caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
332     ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
333     """
334     return [frames[i : i + frames_in] for i in range(len(frames) - frames_in + 1)]
335
336
337 class SequentialFrameStore:
338     """In-memory sliding buffer over a forward-only frame reader, sized for the
339     detector's sliding-window access pattern (peak O(window) frames, not O(video)).
340
341     The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
342     ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
343     in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
344     start+1, ...``. So the global read sequence dips back by ``window-1`` at each window

```

```

345 boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
346 far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
347 earliest index any not-yet-finished window can still ask for) and decode each source
348 frame exactly once via the forward-only ``reader(index)``.
349 """
350
351 def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
352     self._reader = reader
353     self._total = int(total)
354     self._window = max(1, int(window))
355     self._buf: dict = {}
356     # On a resumed run the detector's first needed frame is `start_index`; begin
357     # pulling there so the reader can skip (seek past) the already-cached prefix
358     # instead of decoding 0..start_index-1 just to throw them away.
359     self._next = int(start_index) # next index not yet pulled from the reader
360     self._max_seen = int(start_index) - 1
361     self._floor = int(start_index) # lowest index we still keep (never evict below)
362
363 def get(self, index: int):
364     index = int(index)
365     if index < self._floor:
366         raise KeyError(
367             f"frame {index} already evicted (below floor {self._floor}); the access
368             "
369             f"pattern must stay within a window of the highest frame seen"
370         )
371     if index >= self._total:
372         raise KeyError(f"frame {index} out of range (total {self._total})")
373     # Pull forward from the reader until the requested index is buffered.
374     while self._next <= index:
375         self._buf[self._next] = self._reader(self._next)
376         self._next += 1
377     if index > self._max_seen:
378         self._max_seen = index
379     # Evict frames older than the earliest a still-running window could re-request:
380     # once we've seen index `m`, no future window starts before `m - (window-1)`.
381     new_floor = max(self._floor, self._max_seen - (self._window - 1))
382     for k in range(self._floor, new_floor):
383         self._buf.pop(k, None)
384     self._floor = new_floor
385     return self._buf[index]
386
387 def _index_from_synthetic_path(path: str) -> int:
388     """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
389
390     Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
391     in lockstep with the on-disk naming scheme.
392     """
393     return _frame_id(path)
394
395
396 def _bgr_to_pil_rgb(frame_bgr):
397     """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
398     EXACT
399     PIL RGB image WASB's disk path produces.
400
401     The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
402     Image.open(PNG).convert('RGB')``;
403     because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
404     COLOR_BGR2RGB)``
405     bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
406     streamed
407     frame indistinguishable from the on-disk one to everything downstream.
408     """

```

```

405     import cv2
406     from PIL import Image
407
408     return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
409
410
411     def _make_streamed_read_image(store, real_read_image):
412         """Build the ``read_image`` replacement used during a streaming detector pass.
413
414         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB
415         image,
416         matching ``read_image``'s real return type); any path that doesn't parse to an index
417         falls
418         through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
419         testable
420         without the WSL-only ``dataloaders`` package.
421         """
422
423         def _read_image(path, *args, **kwargs):
424             idx = _index_from_synthetic_path(path)
425             if idx < 0:
426                 return real_read_image(path, *args, **kwargs)
427             return _bgr_to_pil_rgb(store.get(idx))
428
429         return _read_image
430
431     @contextmanager
432     def _streaming_read_image_patch(
433         frame_loader, total: int, window: int = 1, start_index: int = 0
434     ):
435         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
436         decoded
437         frames during the detector pass, byte-for-byte as it would over real PNGs.
438
439         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
440         ``cv2.imread``. ``read_image`` (``utils.utils``) does
441         ``Image.open(path).convert('RGB')``,
442         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
443         feed it a
444         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
445         byte-identical to the disk round-trip, and the shim never touches the filesystem so
446         the
447         ``osp.exists`` guard is sidestepped for synthetic stream paths.
448
449         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
450         actually
451         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
452         in that
453         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
454         imported
455         reference).
456
457         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
458         wrapped
459         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
460         on a
461         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
462         The
463         original reader is restored on exit.
464         """
465         from dataloaders import dataset_loader as _dl
466
467         store = SequentialFrameStore(
468             frame_loader, total, window=window, start_index=start_index

```



```

457         )
458         real_read_image = _dl.read_image
459         # Rebind read_image in the consuming module for the duration of the detector pass;
460         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
461         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
462         try:
463             yield store
464         finally:
465             _dl.read_image = real_read_image # type: ignore[assignment]
466
467
468         # ----- #
469         # Inference
470         # ----- #
471         class _PrefetchIter:
472             """Bounded producer-thread wrapper over a batch iterator (the num_workers=0
DataLoader).
473
474             In streaming mode the DataLoader must stay single-process (the in-memory frame store
+
475             the cv2.imread shim live in this process and cannot be pickled to workers), so the
476             CPU-side batch assembly (decode + PIL + ToTensor) SERIALIZES with GPU inference ?
477             measured on the first real L4 serving run (2026-07-03) this left the GPU <1%
utilized.
478             A single producer THREAD fixes the overlap without multiprocessing: cv2/PIL/numpy
479             release the GIL for the heavy parts, the store's forward-only access pattern is
480             preserved (one consumer thread iterating sequentially), and the bounded queue caps
481             memory at ``depth`` batches. A producer exception is re-raised in the consumer."""
482
483             _END = object()
484
485             def __init__(self, src, depth: int = 2):
486                 import queue
487                 import threading
488
489                 self._q: "queue.Queue" = queue.Queue(maxsize=max(1, int(depth)))
490                 self._exc: "BaseException | None" = None
491
492             def _fill() -> None:
493                 try:
494                     for item in src:
495                         self._q.put(item)
496                     except BaseException as e: # noqa: BLE001 - surfaced in __next__
497                         self._exc = e
498                     finally:
499                         self._q.put(self._END)
500
501                 self._t = threading.Thread(target=_fill, name="wasb-prefetch", daemon=True)
502                 self._t.start()
503
504             def __iter__(self):
505                 return self
506
507             def __next__(self):
508                 item = self._q.get()
509                 if item is self._END:
510                     if self._exc is not None:
511                         raise self._exc
512                     raise StopIteration
513                 return item
514
515
516         def run(

```

```

517     frames,
518     weights: str,
519     out_csv: str,
520     sport: str = "badminton",
521     limit: int = 0,
522     batch_size: int = 8,
523     num_workers: int = 4,
524     log_every_batches: int = 50,
525     cache_dir: "str | None" = None,
526     flush_every: int = 1000,
527     fresh: bool = False,
528     frame_loader=None,
529     source_key: "str | None" = None,
530     device: str = "cuda",
531     prefetch_batches: int = 0,
532 ) -> str:
533     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
534
535     Two equivalent ways to supply pixels (output is bit-identical between them):
536
537     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
538       globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
539       ``cv2.imread``.
540
541     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)

```

16. user (2026-07-07T04:05:35.376Z)

```

540     frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
541     pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
542     reader, which returns a PIL RGB image) over the DataLoader pass so every other
543     line of
544     ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
545     identical
546     to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
547     ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces
548     ``num_workers=0``
549     (the in-process shim + buffer can't cross worker processes).
550
551     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
552     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
553     after reboot still matches.
554     """
555     import time
556
557     import torch
558     from dataloaders import build_img_transforms, build_seq_transforms
559     from dataloaders.dataset_loader import ImageDataset
560     from torch.utils.data import DataLoader
561     from utils import Center
562
563     from detectors import build_detector
564
565     _ensure_np_inf() # NumPy-2.x env: the tracker pass reads np.Inf
566     streaming = frame_loader is not None
567
568     cfg = _build_cfg(weights, sport, device)
569     frames_in = int(cfg["model"]["frames_in"])
570     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
571     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
572
573     if streaming:
574         # ``frames`` is already the ordered list of (synthetic) frame paths.
575         if not isinstance(frames, (list, tuple)):
576             raise SystemExit(
577                 "streaming mode requires ``frames`` to be a list of frame paths"

```

```

575         )
576         frames = list(frames)
577         frames_dir = source_key or "stream"
578     else:
579         frames_dir = frames
580         frames = sorted(
581             glob.glob(osp.join(frames_dir, "*.png"))
582             + glob.glob(osp.join(frames_dir, "*.jpg"))
583         )
584     if limit:
585         frames = frames[:limit]
586     if len(frames) < frames_in:
587         where = frames_dir if not streaming else "video stream"
588         raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
589
590     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
591     checkpoint).
592     samples = []
593     for window in build_windows(frames, frames_in):
594         annos = [
595             {"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
596             for p in window
597         ]
598         samples.append({"frames": window, "annos": annos})
599     windows_total = len(samples)
600
601     # ---- cache setup / resume decision ----- #
602     if cache_dir is None:
603         cache_dir = default_cache_dir(out_csv)
604         os.makedirs(cache_dir, exist_ok=True)
605         det_jsonl, _ = _cache_paths(cache_dir)
606         cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
607         manifest = None if fresh else read_manifest(cache_dir)
608         resume = manifest_compatible(
609             manifest or {},
610             frames_dir=cache_key,
611             weights=weights,
612             sport=sport,
613             frames_in=frames_in,
614             frames_total=len(frames),
615             device=device,
616             decode_backend="cpu", # run() IS the cpu decode path (disk frames / cv2 stream)
617         )
618     if resume:
619         windows_done, det_by_fid = load_and_clean_cache(cache_dir)
620         logger.info(
621             f"resuming from cache: {windows_done}/{windows_total} windows already
622             detected"
623         )
624     else:
625         if manifest is not None:
626             logger.info("cache incompatible with this run -> starting fresh")
627             reset_cache(cache_dir)
628             windows_done, det_by_fid = 0, defaultdict(list)
629
630     base_manifest = {
631         "version": CACHE_VERSION,
632         "frames_dir": cache_key,
633         "weights": weights,
634         "sport": sport,
635         "frames_in": frames_in,
636         "frames_total": len(frames),
637         "device": device,
638         "decode_backend": "cpu",
639         "windows_total": windows_total,

```

```

638         "windows_done": windows_done,
639     }
640     write_manifest(cache_dir, base_manifest)
641
642     # ---- detector pass over the un-cached windows ----- #
643     remaining = samples[windows_done:]
644     if remaining:
645         _, transform_test = build_img_transforms(cfg)
646         try:
647             _, seq_transform_test = build_seq_transforms(cfg)
648         except Exception:
649             seq_transform_test = None
650         ds = ImageDataset(
651             cfg,
652             remaining,
653             input_wh,
654             output_wh,
655             transform=transform_test,
656             seq_transform=seq_transform_test,
657             is_train=False,
658         )
659         # Streaming forces single-process loading: the in-memory frame store + the
660         # cv2.imread shim live in THIS process and can't be pickled to DataLoader
661         loader_workers = 0 if streaming else num_workers
662         loader = DataLoader(
663             ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers
664         )
665         with _torch_load_compat(): # torch-2.6 env: WASB's torch.load needs
666             detector = build_detector(cfg)
667
668         from contextlib import ExitStack
669
670         t0 = time.time()
671         done = windows_done
672         win_idx = windows_done
673         log_every = max(1, log_every_batches)
674         flush_n = max(1, flush_every)
675         pending = []
676         logger.info(
677             f"detecting on {len(remaining)} remaining windows "
678             f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
679             f"source={'video-stream' if streaming else 'frames-dir'})..."
680         )
681         det_f = open(det_jsonl, "a")
682         try:
683             with ExitStack() as stack:
684                 stack.enter_context(torch.no_grad())
685                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
686                 # in-memory decoded frame for that synthetic path; every other line of
687                 # `__getitem__` runs unchanged so the tensor matches the PNG round-trip
688                 # exactly.
689                 # The detector's first needed frame is `windows_done` (window i starts
690                 # at
691                 # frame i), so prime the store there for a resume.
692                 if streaming:
693                     stack.enter_context(
694                         _streaming_read_image_patch(
695                             frame_loader,
696                             len(frames),
697                             window=frames_in,
698                             start_index=windows_done,
699                         )

```

```

698         )
699         # Overlap CPU batch assembly with GPU inference (streaming/workers=0
only ?
700         # a multi-worker DataLoader already overlaps via its worker processes).
701         batches = (
702             _PrefetchIter(loader, depth=prefetch_batches)
703             if prefetch_batches and loader_workers == 0
704             else loader
705         )
706         for bi, batch in enumerate(batches):
707             imgs, _hms, trans = batch[0], batch[1], batch[2]
708             img_paths = [
709                 list(t) for t in batch[-1]
710             ] # [frame_in][batch] -> path
711             batch_results, _ = detector.run_tensor(imgs, trans)
712             nb = imgs.shape[0]
713             for ib in range(nb):
714                 frames_payload = []
715                 for ie in sorted(batch_results[ib].keys()):
716                     p = img_paths[ie][ib]
717                     fid = _frame_id(p)
718                     plain = [_det_plain(d) for d in batch_results[ib][ie]]
719                     frames_payload.append([fid, plain])
720                     det_by_fid[fid].extend(plain)
721                 pending.append({"w": win_idx, "f": frames_payload})
722                 win_idx += 1
723             done += nb
724             if len(pending) >= flush_n or done >= windows_total:
725                 _append_windows(det_f, pending)
726                 pending = []
727                 base_manifest["windows_done"] = done
728                 write_manifest(cache_dir, base_manifest)
729             if (bi + 1) % log_every == 0 or done >= windows_total:
730                 el = time.time() - t0
731                 new_done = done - windows_done
732                 rate = new_done / el if el > 0 else 0.0
733                 eta = (windows_total - done) / rate if rate > 0 else 0.0
734                 logger.info(
735                     f"{done}/{windows_total} "
736                     f"({100 * done // max(1, windows_total)}%) | {rate:.1f}
win/s | "
737                     f"elapsed {el:.0f}s | ETA {eta:.0f}s"
738                 )
739             if pending:
740                 _append_windows(det_f, pending)
741                 base_manifest["windows_done"] = done
742                 write_manifest(cache_dir, base_manifest)
743         finally:
744             det_f.close()
745             logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
746     else:
747         logger.info("detector cache already complete -> tracker-only re-run")
748
749     # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
750     return _tracker_pass_and_write(cfg, det_by_fid, out_csv)
751
752
753 def _tracker_pass_and_write(cfg, det_by_fid, out_csv: str) -> str:
754     """Tracker re-run over the full ordered detection cache ? trajectory CSV.
755
756     Shared verbatim by every decode backend: the tracker consumes the same per-frame
757     candidate lists regardless of how the pixels were decoded, so keeping this single
758     keeps the output contract single.
759
760     NB: this is WASB's OWN local `trackers` module (from its repo, on sys.path inside

```

```

the
761     `wasb` conda env), NOT the Roboflow `trackers` package. The `type: ignore`
suppresses
762     the mypy false positive where the main env resolves `trackers` to the Roboflow
package
763     (which has no `build_tracker`). There is NO runtime collision ? this runs in a
separate
764     subprocess/conda env. Do NOT "clean up" the ignore without removing the Roboflow
dep.
765     """
766     from trackers import build_tracker # type: ignore[attr-defined]
767
768     _ensure_np_inf() # OnlineTracker reads np.Inf (removed in NumPy 2.0)
769     tracker = build_tracker(cfg)
770     tracker.refresh()
771     rows = []
772     for fid in sorted(det_by_fid.keys()):
773         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
774         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
775
776     _atomic_write_trajectory(out_csv, rows)
777     visible = sum(1 for _, v, *_ in rows if v)
778     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
779     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
780     return out_csv
781
782
783 def extract_frames(video_path: str, frames_dir: str) -> str:
784     """Decode a video to PNG frames named by source frame index (00000000.png ...).
785
786     Idempotent: if the dir already holds the expected number of frames (matching the
787     container's reported frame count) we skip re-decoding ? re-extraction after a
788     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
789     extracting with ffmpeg on the host and passing --frames_dir.
790     """
791     import cv2
792
793     os.makedirs(frames_dir, exist_ok=True)
794     cap = cv2.VideoCapture(video_path)
795     if not cap.isOpened():
796         raise SystemExit(f"cannot open video: {video_path}")
797     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
798     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
799     if expected > 0 and existing >= expected:
800         cap.release()
801         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
802         return frames_dir
803     i = 0
804     while True:
805         ok, frame = cap.read()
806         if not ok:
807             break
808         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
809         i += 1
810     cap.release()
811     logger.info(f"extracted {i} frames -> {frames_dir}")
812     return frames_dir
813
814
815 def _probe_frame_count(video_path: str) -> int:
816     """Container-reported frame count via cv2 (0 if unknown). Used to size the
stream."""
817     import cv2
818
819     cap = cv2.VideoCapture(video_path)

```

```

820     if not cap.isOpened():
821         raise SystemExit(f"cannot open video: {video_path}")
822     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
823     cap.release()
824     return n
825
826
827 def make_video_frame_loader(video_path: str):
828     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
829
830     ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
831     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
832     detector's sliding-window order); it reads sequentially and only uses a frame-peek
833     to
834     skip forward when a *resume* starts past the current position. Each ``cap.read()``
835     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG
836     would
837     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
838
839     Returns the count hint from the container so the caller can build the frame list;
840     ``release`` closes the capture.
841     """
842     import cv2
843
844     cap = cv2.VideoCapture(video_path)
845     if not cap.isOpened():
846         raise SystemExit(f"cannot open video: {video_path}")
847     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
848     state = {"pos": 0} # next index cap.read() will return
849
850     def frame_loader(index: int):
851         index = int(index)
852         if index < state["pos"]:
853             raise RuntimeError(
854                 f"streaming decode is forward-only; got index {index} after
855                 {state['pos']}")
856         if index > state["pos"]:
857             # Forward skip only happens when resuming past cached windows. Seek once.
858             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
859             state["pos"] = index
860             ok, frame = cap.read()
861             if not ok:
862                 raise RuntimeError(f"failed to decode frame {index} from {video_path}")
863             state["pos"] += 1
864             return frame
865
866     def release():
867         cap.release()
868
869     return frame_loader, count_hint, release
870
871 def run_video_streaming(
872     video_path: str,
873     weights: str,
874     out_csv: str,
875     sport: str = "badminton",
876     limit: int = 0,
877     batch_size: int = 8,
878     log_every_batches: int = 50,
879     cache_dir: "str | None" = None,
880     flush_every: int = 1000,
881     fresh: bool = False,
882     device: str = "cuda",

```

```

882         prefetch_batches: int = 0,
883     ) -> str:
884         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
885         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
886         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
887         ``run``'s docstring for why.
888         """
889         frame_loader, count_hint, release = make_video_frame_loader(video_path)
890         try:
891             if count_hint <= 0:
892                 raise SystemExit(
893                     f"could not determine frame count for {video_path}; cannot stream "
894                     f"(fall back to frame extraction with --frames_dir)"
895                 )
896             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
897             source_key = "video:" + osp.abspath(video_path)
898             return run(
899                 frame_list,
900                 weights,
901                 out_csv,
902                 sport,
903                 limit,
904                 batch_size=batch_size,
905                 num_workers=0,
906                 log_every_batches=log_every_batches,
907                 cache_dir=cache_dir,
908                 flush_every=flush_every,
909                 fresh=fresh,
910                 frame_loader=frame_loader,
911                 source_key=source_key,
912                 device=device,
913                 prefetch_batches=prefetch_batches,
914             )
915         finally:
916             release()
917
918
919     # ----- #

```

17. assistant (2026-07-07T04:06:08.580Z)

Now the untracked test file and the presets.

18. user (2026-07-07T04:06:10.026Z)

```

1     """GPU-resident NVDEC decode backend (#506) ? the parts testable off-GPU, plus the
2     on-box parity gate.
3
4     THE lesson this file locks (issue #506): the GPU resize MUST replicate WASB's
5     ``cv2.warpAffine(get_affine_transform(...), INTER_LINEAR)`` exactly. A naive
6     ``F.interpolate`` resize samples at half-pixel-convention positions ? a constant
7     sub-pixel shift versus cv2's integer-pixel-center affine ? which tips the sigmoid
8     past ``score_threshold`` on marginal frames and silently over-detects (rally F1
9     0.58 ? 0.07 on the hard golden clip GX010128, while the easy clip masked it at 98%
10    trajectory agreement). The grid-parity tests below fail if anyone "simplifies" the
11    affine ``grid_sample`` back to a plain resize.
12
13    The trajectory-parity contract itself (``nvdec_resident`` ? ``cpu`` on a real clip)
14    is a HARDWARE test ? it needs a CUDA GPU, PyNvVideoCodec, the WASB repo, weights and
15    a fixture clip ? so it skips everywhere else (CI is CPU-only). See
16    ``test_nvdec_resident_trajectory_matches_cpu_on_fixture`` for how to run it on the box.
17    """
18
19    import csv
20    import importlib.util

```



```

21 import os
22 import os.path as osp
23 import sys
24
25 import numpy as np
26 import pytest
27
28 from backend.pipeline.detectors import wasb_infer as wi
29
30
31 # ----- #
32 # WASB's exact affine geometry, replicated for the tests (utils/image.py
33 # ``get_affine_transform`` with rot=0: center = frame middle, scale = max(h, w),
34 # 3-point correspondence through cv2.getAffineTransform). The runtime path calls
35 # WASB's own function; the tests only need matrices of the same shape.
36 # ----- #
37 def _get_3rd_point(a, b):
38     direct = a - b
39     return b + np.array([-direct[1], direct[0]], dtype=np.float32)
40
41
42 def _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h):
43     """(forward, inverse) 2x3 matrices exactly as WASB's get_transform builds them."""
44     import cv2
45
46     c = np.array([orig_w / 2.0, orig_h / 2.0], dtype=np.float32)
47     s = max(orig_h, orig_w) * 1.0
48     src = np.zeros((3, 2), dtype=np.float32)
49     dst = np.zeros((3, 2), dtype=np.float32)
50     src[0, :] = c
51     src[1, :] = c + np.array([0, -s * 0.5], dtype=np.float32)
52     dst[0, :] = [dst_w * 0.5, dst_h * 0.5]
53     dst[1, :] = dst[0, :] + np.array([0, -dst_w * 0.5], dtype=np.float32)
54     src[2, :] = _get_3rd_point(src[0, :], src[1, :])
55     dst[2, :] = _get_3rd_point(dst[0, :], dst[1, :])
56     fwd = cv2.getAffineTransform(np.float32(src), np.float32(dst))
57     inv = cv2.getAffineTransform(np.float32(dst), np.float32(src))
58     return fwd, inv
59
60
61 def _smooth_test_image(orig_w, orig_h, seed=506):
62     """Deterministic smooth float32 RGB image: smooth enough that both bilinear
63     implementations agree on identical sample positions, textured enough that a
64     sub-pixel sampling shift shows up loudly."""
65     import cv2
66
67     rng = np.random.default_rng(seed)
68     img = rng.random((orig_h, orig_w, 3)).astype(np.float32)
69     return cv2.GaussianBlur(img, (0, 0), sigmaX=2.0)
70
71
72 # ----- #
73 # The resize-fidelity lock (CPU-runnable: torch + cv2, no GPU / no WASB repo)
74 # ----- #
75 def test_affine_grid_matches_cv2_warpAffine():
76     """grid_sample over build_affine_grid == cv2.warpAffine on WASB's production
77     geometry (16:9 frame, the badminton 512x288-shaped target)."""
78     torch = pytest.importorskip("torch")
79     cv2 = pytest.importorskip("cv2")
80     import torch.nn.functional as F
81
82     orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36 # 16:9 ? 16:9, scale 3.0
83     img = _smooth_test_image(orig_w, orig_h)
84     fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
85

```

```

86     ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
87
88     grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
89     assert tuple(grid.shape) == (1, dst_h, dst_w, 2)
90     chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
91     got = (
92         F.grid_sample(
93             chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
94         )
95         .squeeze(0)
96         .permute(1, 2, 0)
97         .numpy()
98     )
99
100     diff = np.abs(got - ref)
101     # cv2 quantizes bilinear coefficients (INTER_BITS); anything beyond that noise
102     # floor means the geometry diverged. The naive-resize failure below is ~100x this.
103     assert diff.mean() < 1e-3, f"mean |?| {diff.mean():.2e} vs cv2.warpAffine"
104     assert diff.max() < 0.02, f"max |?| {diff.max():.2e} vs cv2.warpAffine"
105
106
107 def test_naive_interpolate_does_not_match_cv2_warpaffine():
108     """The #506 regression signature: F.interpolate is NOT the affine warp. If this
109     test ever fails (interpolate suddenly 'matches'), the reference setup is broken ?
110     it exists so the grid can never be quietly replaced by a plain resize."""
111     torch = pytest.importorskip("torch")
112     cv2 = pytest.importorskip("cv2")
113     import torch.nn.functional as F
114
115     orig_w, orig_h, dst_w, dst_h = 192, 108, 64, 36
116     img = _smooth_test_image(orig_w, orig_h)
117     fwd, inv = _wasb_affine_matrices(orig_w, orig_h, dst_w, dst_h)
118
119     ref = cv2.warpAffine(img, fwd, (dst_w, dst_h), flags=cv2.INTER_LINEAR)
120     chw = torch.from_numpy(img).permute(2, 0, 1).unsqueeze(0)
121
122     grid = wi.build_affine_grid(inv, dst_w, dst_h, orig_w, orig_h)
123     affine = (
124         F.grid_sample(
125             chw, grid, mode="bilinear", align_corners=False, padding_mode="border"
126         )
127         .squeeze(0)
128         .permute(1, 2, 0)
129         .numpy()
130     )
131     naive = (
132         F.interpolate(chw, size=(dst_h, dst_w), mode="bilinear", align_corners=False)
133         .squeeze(0)
134         .permute(1, 2, 0)
135         .numpy()
136     )
137
138     affine_err = np.abs(affine - ref).mean()
139     naive_err = np.abs(naive - ref).mean()
140     # The half-pixel-convention shift is a real, systematic error ? far above the
141     # affine grid's coefficient-quantization noise floor.
142     assert naive_err > 1e-3, f"naive resize unexpectedly matches cv2 ({naive_err:.2e})"
143     assert naive_err > 5 * affine_err, (
144         f"affine grid ({affine_err:.2e}) must beat naive resize ({naive_err:.2e}) "
145         f"decisively ? otherwise the #506 lesson is unlearned"
146     )
147
148
149 # ----- #
150 # Cache keying: nvdec detections are trajectory-level (not byte-level) equivalents

```

```

151 # of cpu ones, so the resumable cache must never be shared across backends.
152 # ----- #
153 def _manifest(**over):
154     m = {
155         "version": wi.CACHE_VERSION,
156         "frames_dir": "video:/x.mp4",
157         "weights": "/w.pth.tar",
158         "sport": "badminton",
159         "frames_in": 3,
160         "frames_total": 100,
161         "device": "cuda",
162     }
163     m.update(over)
164     return m
165
166
167 def test_cache_never_shared_across_decode_backends():
168     kw = dict(
169         frames_dir="video:/x.mp4",
170         weights="/w.pth.tar",
171         sport="badminton",
172         frames_in=3,
173         frames_total=100,
174         device="cuda",
175     )
176     # Pre-#506 manifests carry no decode_backend field == cpu-decoded: still valid
177     # for a cpu run (no invalidation on upgrade), NEVER for an nvdec run.
178     legacy = _manifest()
179     assert wi.manifest_compatible(legacy, **kw)
180     assert not wi.manifest_compatible(legacy, **kw, decode_backend="nvdec_resident")
181     # And an nvdec cache must not be replayed by a cpu run.
182     nv = _manifest(decode_backend="nvdec_resident")
183     assert wi.manifest_compatible(nv, **kw, decode_backend="nvdec_resident")
184     assert not wi.manifest_compatible(nv, **kw)
185
186
187 # ----- #
188 # CLI guards: the nvdec branch must fail loud BEFORE any torch/GPU work.
189 # ----- #
190 def test_cli_nvdec_resident_requires_video(monkeypatch, tmp_path):
191     monkeypatch.setattr(
192         sys,
193         "argv",
194         [
195             "wasb_infer.py",
196             "--decode-backend",
197             "nvdec_resident",
198             "--frames_dir",
199             str(tmp_path),
200             "--weights",
201             "w.pth.tar",
202             "--out",
203             str(tmp_path / "o.csv"),
204         ],
205     )
206     with pytest.raises(SystemExit, match="requires --video"):
207         wi.main()
208
209
210 def test_cli_nvdec_resident_requires_cuda_device(monkeypatch, tmp_path):
211     monkeypatch.setattr(
212         sys,
213         "argv",
214         [
215             "wasb_infer.py",

```

```

216         "--decode-backend",
217         "nvdec_resident",
218         "--video",
219         "v.mp4",
220         "--device",
221         "cpu",
222         "--weights",
223         "w.pth.tar",
224         "--out",
225         str(tmp_path / "o.csv"),
226     ],
227 )
228 with pytest.raises(SystemExit, match="cuda"):
229     wi.main()
230
231
232 def test_cli_rejects_unknown_decode_backend(monkeypatch, tmp_path, capsys):
233     monkeypatch.setattr(
234         sys,
235         "argv",
236         [
237             "wasb_infer.py",
238             "--decode-backend",
239             "gpu", # not a backend; argparse choices reject it
240             "--video",
241             "v.mp4",
242             "--weights",
243             "w.pth.tar",
244             "--out",
245             str(tmp_path / "o.csv"),
246         ],
247     )
248     with pytest.raises(SystemExit):
249         wi.main()
250     assert "invalid choice" in capsys.readouterr().err
251
252
253 # ----- #
254 # The #506 unit gate: nvdec_resident trajectory ? cpu trajectory on a real clip.
255 # HARDWARE test ? skips unless the box provides everything. Run on the GPU box
256 # (inside the torch-2.x wasb env, which has torch+cuda, PyNvVideoCodec, cv2):
257 #
258 # WASB_REPO_DIR=~/.models/WASB-SBDT \
259 # WASB_WEIGHTS=~/.models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
260 # WASB_PARITY_VIDEO=/path/to/short_fixture_clip.mp4 \
261 # python -m pytest tests/test_wasb_nvdec_resident.py -k parity -v
262 # ----- #
263 def _load_traj(path):
264     out = {}
265     with open(path, newline="") as f:
266         for row in csv.DictReader(f):
267             out[int(float(row["Frame"]))] = (
268                 int(float(row["Visibility"])),
269                 float(row["X"]),
270                 float(row["Y"]),
271             )
272     return out
273
274
275 def test_nvdec_resident_trajectory_matches_cpu_on_fixture(tmp_path):
276     repo = osp.expanduser(os.environ.get("WASB_REPO_DIR", ""))
277     weights = osp.expanduser(os.environ.get("WASB_WEIGHTS", ""))
278     video = osp.expanduser(os.environ.get("WASB_PARITY_VIDEO", ""))
279     if not (repo and weights and video):
280         pytest.skip("set WASB_REPO_DIR + WASB_WEIGHTS + WASB_PARITY_VIDEO to run")

```

```

281     for p in (osp.join(repo, "src"), weights, video):
282         if not osp.exists(p):
283             pytest.skip(f"missing {p}")
284     torch = pytest.importorskip("torch")
285     if not torch.cuda.is_available():
286         pytest.skip("needs a CUDA GPU")
287     pytest.importorskip("PyNvVideoCodec")
288
289     # Stage wasb_infer.py into the WASB repo src/ (exactly as the runners do) and import
290     # THAT copy, so its WASB imports and Hydra's configs/ dir resolve.
291     import shutil
292
293     from backend.pipeline.detectors import native_wasb_runner as nwr
294
295     src = osp.join(repo, "src")
296     staged = osp.join(src, "wasb_infer.py")
297     shutil.copy(nwr._INFER_PY, staged)
298     sys.path.insert(0, src)
299     try:
300         spec = importlib.util.spec_from_file_location("wasb_infer_staged", staged)
301         mod = importlib.util.module_from_spec(spec)
302         assert spec.loader is not None
303         spec.loader.exec_module(mod)
304
305         cpu_csv = str(tmp_path / "cpu_traj.csv")
306         gpu_csv = str(tmp_path / "gpu_traj.csv")
307         mod.run_video_streaming(video, weights, cpu_csv, fresh=True)
308         mod.run_video_nvdec_resident(video, weights, gpu_csv, fresh=True)
309
310         cpu, gpu = _load_traj(cpu_csv), _load_traj(gpu_csv)
311         common = sorted(set(cpu) & set(gpu))
312         assert len(common) >= 10, "fixture too short to be a meaningful gate"
313         agree = sum(1 for f in common if cpu[f][0] == gpu[f][0]) / len(common)
314         dists = [
315             float(np.hypot(cpu[f][1] - gpu[f][1], cpu[f][2] - gpu[f][2]))
316             for f in common
317             if cpu[f][0] == 1 and gpu[f][0] == 1
318         ]
319         # Validated on the L4 (#506): 96.3% visibility agreement, median 0.036 px.
320         assert agree >= 0.95, f"visibility agreement {agree:.3f} < 0.95"
321         assert dists, "no frames visible in both trajectories"
322         assert float(np.median(dists)) <= 1.0, (
323             f"median both-visible distance {float(np.median(dists)):.3f}px > 1.0px"
324         )
325     finally:
326         sys.path.remove(src)
327

```

19. user (2026-07-07T04:06:10.596Z)

```

1     """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3     A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4     config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5     knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7     Precedence (lowest ? highest), realised by the loader::
8
9         built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no

```

```

14     profile selected the loader is byte-identical to the pre-M1 behaviour (`profile=""` ==
15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     ""
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
52             "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53             # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54             # wasb GPU-feed tuning: the parity defaults (batch 8, no prefetch) left the paid
L4
55             # <1% utilized on the first real serving run (2026-07-03) ? the pipeline was
56             # CPU-feed-bound. Serving opts into the measured fast values; local/WSL
unchanged.
57             "indexing": {
58                 "detector_impl": "native",
59                 "skip_ai_handoff": False,
60                 "wasb": {"batch_size": 64, "prefetch_batches": 4},
61             },
62             "storage": {"backend": "gcs"},
63         },
64         "cpu-mock": {
65             "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
66             # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.

```

```

67         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
68         "storage": {"backend": "local"},
69     },
70 }
71
72
73 def is_known_profile(profile: str) -> bool:
74     """True for a non-empty, recognised profile name."""
75     return bool(profile) and profile in PROFILE_PRESETS
76
77
78 def preset_for(profile: str) -> dict[str, Any]:
79     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
80     module-level table), or ``{}`` for ``""`` / an unknown profile."""
81     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
82
83
84 def _truthy(val: Optional[str]) -> bool:
85     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
86     if val is None:
87         return False
88     return val.strip().lower() in {"1", "true", "yes", "on"}
89
90
91 def _cuda_available() -> bool:
92     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
93     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
94     try:
95         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
96         # and optional). Only reached when a caller opts into probe_cuda.
97         import torch
98
99         return bool(torch.cuda.is_available())
100     except Exception:
101         return False
102
103
104 def probe_cuda_available() -> bool:
105     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
106
107     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
108     that *wants* the GPU heads-up can pass the result in without the startup module
importing
109     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
110     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
111     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
112     error that could false-fail."""
113     return _cuda_available()
114
115
116 def suggest_profile(
117     env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
118 ) -> str:
119     """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
120
121     Signals, in priority order:
122

```

```

123         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
124         - ``CI`` truthy                             ? ``"cpu-mock"``
125         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
126         - otherwise                                 ? ``""`` (no strong suggestion)
127
128     ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
never
129     imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
passes
130     ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
on it.
131     """
132     e = os.environ if env is None else env
133     if e.get("K_SERVICE"):
134         return "cloud-serving"
135     if _truthy(e.get("CI")):
136         return "cpu-mock"
137     if probe_cuda and _cuda_available():
138         return "truly-local"
139     return ""
140

```

20. assistant (2026-07-07T04:09:12.065Z)

Let me verify the healthcheck gating in trajectory_hybrid and check `\_ok\_cfg`/`\_drive\_success` handling in the runner tests.

21. user (2026-07-07T04:09:13.271Z)

```

270         ai_max_workers = idx_cfg.ai_max_workers
271         log_candidates = idx_cfg.log_yolo_candidates
272         store_candidates = idx_cfg.store_yolo_candidates
273
274         detection_params = {
275             "detector": self.name,
276             "velocity_thresh": velocity_thresh,
277             "merge_gap": merge_gap,
278             "min_action_window_duration": min_window_duration,
279             **runner_params,
280         }
281
282         # --- Phase 1: env healthcheck (fail fast with a clear message) ---
283         ok, msg = runner.healthcheck()
284         if not ok:
285             logger.error("%s detector environment not ready: %s", label, msg)
286             # A detector FAILURE, not a 0-rally video ? surface it so the caller marks
the run
287             # failed/retryable instead of writing an empty-but-"successful" result.
288             return Failure(
289                 message=f"{label} detector environment not ready: {msg}",
290                 is_recoverable=True,
291             )
292         logger.info("%s detector env OK: %s", label, msg)
293
294         # --- Phase 2: trajectory -> candidate rally windows ---
295         # Trajectory detectors process the whole video in one pass (they carry
296         # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
297         t_start = time.time()
298         out_dir = os.path.join(output_base(self.config), self.family)
299         csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
300         if not csv_path:
301             logger.error("%s produced no trajectory; aborting.", label)
302             # The cold-start / timeout failure mode (e.g. WASB native inference timed
out after
303             # timeout_sec, or the detector env yielded nothing). This is a DETECTOR

```



```
failure ? NOT
304      # a legitimately empty match ? so return a Failure the worker marks as
failed/retryable
305      # rather than a silent 0-segment "success" (the wf294 incident).
306      return Failure(
307          message=(
308              f"{label} detector produced no trajectory (timed out or aborted) ? "
309              "detector failure, not a 0-rally video."
```

22. user (2026-07-07T04:10:28.448Z)

Structured output provided successfully